Predicting Vehicle Fuel Efficiency Using Machine Learning

MSc Data Science Final Project
University of Hertfordshire

Student: Siddhartha Ragipani
SRN: 24074708
Supervisor: Sunina Sharvy
Github: https://github.com/ragipanisiddhartha/auto-mpg-fuel-efficiency-ml

Project overview

This notebook implements the analysis and modelling pipeline for predicting vehicle fuel efficiency (mpg) using the Auto MPG dataset from the UCI Machine Learning Repository.

Research question

How accurately can fuel efficiency (mpg) be predicted from vehicle attributes such as cylinders, displacement, horsepower, weight, acceleration, model year and origin, and which regression model provides the best predictive performance on the Auto MPG dataset?

Workflow

The notebook follows these steps:

1. Data loading and cleaning
2. Exploratory data analysis
3. Feature selection and train-test split
4. Linear Regression, Ridge and Lasso modelling
5. Decision Tree and Random Forest modelling
6. Hyperparameter tuning and evaluation
7. Residual analysis and feature importance interpretation

Problem Statement

The aim of this project is to **predict automobile fuel efficiency** (miles per gallon, mpg) from a set of technical vehicle attributes. By building data-driven regression models, the goal is to understand how design choices such as engine size, power and weight affect fuel consumption,

and to provide a tool that can support decision-making when designing new cars. We will implement and compare several machine learning algorithms (including linear and tree-based models) and identify the model that achieves the most accurate and robust mpg predictions.

About the Dataset

This project uses the original **Auto MPG** dataset from the UCI Machine Learning Repository. Each row represents a single car model and includes the following variables:

- **mpg**: Car's fuel efficiency measured in miles per gallon (target variable).
- **cylinders**: Number of cylinders in the car's engine.
- **displacement**: Engine displacement in cubic inches.
- **horsepower**: Engine power output measured in horsepower.
- **weight**: Car's curb weight in pounds.
- **acceleration**: Time taken (in seconds) to accelerate from 0 to 60 mph.
- **model_year**: Model year of the car (coded as 70–82 for 1970–1982).
- **origin**: Coded region of manufacture (e.g. US, Europe, Japan).
- **car_name**: Text label describing the specific car model.

1 – Imports, problem statement, and data loading

1.1 Imports and problem statement

```
# ======================================================
# 1. SETUP
# ======================================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from scipy.stats import ttest_ind, pearsonr
from sklearn.model_selection import train_test_split, cross_val_score,
GridSearchCV
from sklearn.linear_model import LinearRegression, Lasso, Ridge
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score

import plotly.express as px
```

```
import warnings
warnings.filterwarnings("ignore")

plt.style.use("ggplot")
```

Predicting Automobile Fuel Efficiency Using Machine Learning

The goal of this project is to build regression models that predict a car's fuel efficiency, measured in miles per gallon (mpg), from its technical characteristics such as engine size, horsepower, weight, and model year. This helps to understand how design choices influence fuel consumption and to compare the performance of different machine learning algorithms on a well-known benchmark dataset.

```
# ======================================================
# 2. DATA LOADING
# ======================================================

# UCI Auto MPG dataset URL
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/auto-mpg/auto-mpg.data"

# Column names according to UCI documentation
cols = [
    "mpg", "cylinders", "displacement", "horsepower",
    "weight", "acceleration", "model_year", "origin", "car_name"
]

# Load data
auto = pd.read_csv(
    url,
    delim_whitespace=True,
    names=cols,
    on_bad_lines="skip"
)


display(auto.head())
```

{"summary":"{\n \"name\": \"display(auto\",\n \"rows\": 5,\n \"fields\": [\n {\n \"column\": \"mpg\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 1.3038404810405297,\n \"min\": 15.0,\n \"max\": 18.0,\n \"num_unique_values\": 4,\n \"samples\": [\n 15.0,\n 17.0,\n 18.0\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\":

\"cylinders\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 0,\n \"min\": 8,\n \"max\": 8,\n \"num_unique_values\": 1,\n \"samples\": [\n 8\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"displacement\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 19.879637823662684,\n \"min\": 302.0,\n \"max\": 350.0,\n \"num_unique_values\": 5,\n \"samples\": [\n 350.0\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"horsepower\",\n \"properties\": {\n \"dtype\": \"string\",\n \"num_unique_values\": 4,\n \"samples\": [\n \"165.0\"\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"weight\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 110.006817970524,\n \"min\": 3433.0,\n \"max\": 3693.0,\n \"num_unique_values\": 5,\n \"samples\": [\n 3693.0\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"acceleration\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 0.6519202405202649,\n \"min\": 10.5,\n \"max\": 12.0,\n \"num_unique_values\": 4,\n \"samples\": [\n 11.5\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"model_year\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 0,\n \"min\": 70,\n \"max\": 70,\n \"num_unique_values\": 1,\n \"samples\": [\n 70\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"origin\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 0,\n \"min\": 1,\n \"max\": 1,\n \"num_unique_values\": 1,\n \"samples\": [\n 1\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"car_name\",\n \"properties\": {\n \"dtype\": \"string\",\n \"num_unique_values\": 5,\n \"samples\": [\n \"buick skylark 320\"\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n }\n]\n}","type":"dataframe"}

```python
print("Shape:", auto.shape)
```

```
Shape: (398, 9)
```

In this section I imported the main Python libraries used for data analysis and machine learning (pandas, NumPy, seaborn, matplotlib and scikit-learn) and defined the problem I want to solve: predicting car fuel efficiency (mpg) from technical specifications. This clarifies early that the task is a supervised regression problem and justifies why regression models are appropriate. I then loaded the Auto MPG dataset directly from the original UCI Machine Learning Repository using the official column names, which ensures my work is reproducible and clearly linked to an

authoritative data source rather than an already-processed Kaggle copy. The initial head() output confirms that all nine expected columns are present (mpg, cylinders, displacement, horsepower, weight, acceleration, model_year, origin and car_name) and that the data has 398 rows, matching the description of the original dataset.

2. Structure and summary statistics

```
# ======================================================
# 2.1 INITIAL DATA INSPECTION
# ======================================================

print("INFO:")
auto_info = auto.info()
```

```
INFO:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 398 entries, 0 to 397
Data columns (total 9 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   mpg           398 non-null    float64
 1   cylinders     398 non-null    int64
 2   displacement  398 non-null    float64
 3   horsepower    398 non-null    object
 4   weight        398 non-null    float64
 5   acceleration  398 non-null    float64
 6   model_year    398 non-null    int64
 7   origin        398 non-null    int64
 8   car_name      398 non-null    object
dtypes: float64(4), int64(3), object(2)
memory usage: 28.1+ KB
```

```
print("\nDESCRIPTIVE STATISTICS:")
auto_desc = auto.describe()

auto_info, auto_desc
```

```
DESCRIPTIVE STATISTICS:

(None,
               mpg   cylinders  displacement       weight
acceleration  \
 count  398.000000  398.000000    398.000000   398.000000
398.000000
 mean    23.514573    5.454774    193.425879  2970.424623
15.568090
 std      7.815984    1.701004    104.269838   846.841774
2.757689
```

```
 min      9.000000    3.000000     68.000000  1613.000000
8.000000
 25%     17.500000    4.000000    104.250000  2223.750000
13.825000
 50%     23.000000    4.000000    148.500000  2803.500000
15.500000
 75%     29.000000    8.000000    262.000000  3608.000000
17.175000
 max     46.600000    8.000000    455.000000  5140.000000
24.800000

        model_year      origin
 count  398.000000  398.000000
 mean    76.010050    1.572864
 std      3.697627    0.802055
 min     70.000000    1.000000
 25%     73.000000    1.000000
 50%     76.000000    1.000000
 75%     79.000000    2.000000
 max     82.000000    3.000000  )
```

```
auto_desc = auto.describe()
auto_desc
```

{"summary":"{\n \"name\": \"auto_desc\",\n \"rows\": 8,\n \"fields\": [\n {\n \"column\": \"mpg\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 133.37523833494922,\n \"min\": 7.815984312565782,\n \"max\": 398.0,\n \"num_unique_values\": 8,\n \"samples\": [\n 23.514572864321607,\n 23.0,\n 398.0\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"cylinders\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 139.0071020301553,\n \"min\": 1.7010042445332094,\n \"max\": 398.0,\n \"num_unique_values\": 6,\n \"samples\": [\n 398.0,\n 5.454773869346734,\n 8.0\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"displacement\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 143.57617465667641,\n \"min\": 68.0,\n \"max\": 455.0,\n \"num_unique_values\": 8,\n \"samples\": [\n 193.42587939698493,\n 148.5,\n 398.0\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"weight\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 1535.5522437115258,\n \"min\": 398.0,\n \"max\": 5140.0,\n \"num_unique_values\": 8,\n \"samples\": [\n 2970.424623115578,\n 2803.5,\n 398.0\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\

```
n    },\n    {\n      \"column\": \"acceleration\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
135.93788360342714,\n        \"min\": 2.7576889298126757,\n
\"max\": 398.0,\n        \"num_unique_values\": 8,\n
\"samples\": [\n          15.568090452261307,\n          15.5,\n
398.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"model_year\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 120.24225119423292,\n        \"min\":
3.697626646732623,\n        \"max\": 398.0,\n
\"num_unique_values\": 8,\n        \"samples\": [\n
76.01005025125629,\n          76.0,\n          398.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"origin\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\":
140.19214256834297,\n        \"min\": 0.8020548777266163,\n
\"max\": 398.0,\n        \"num_unique_values\": 6,\n
\"samples\": [\n          398.0,\n          1.5728643216080402,\n
3.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\
n}","type":"dataframe","variable_name":"auto_desc"}
```


Here I inspected the overall structure of the dataset using info() and describe() to understand data types, completeness and basic distributions before any modelling. The info() output shows 398 rows and 9 columns, with no missing values reported at this stage and a mix of numeric variables (mpg, cylinders, displacement, weight, acceleration, model_year, origin) and two object-type columns (horsepower and car_name). Seeing horsepower as an object highlights a potential data quality issue, because it should be numeric and may contain non-numeric characters such as question marks for missing values. The descriptive statistics summarise ranges and typical values: mpg varies roughly from 9 to 46, weight from about 1600 to 5100 pounds, and model_year from 1970 to 1982. These summaries confirm that there is substantial variation in both fuel efficiency and car specifications, which is useful for fitting and evaluating predictive models.

3. Data Cleaning and Type Handling

3.1 – Convert horsepower, handle missing values, fix dtypes

```
# ======================================================
# 3. DATA CLEANING AND TYPE HANDLING
# ======================================================

# 3.1 Check missing values before cleaning
print("Missing values BEFORE cleaning:")
print(auto.isna().sum())
```

```
Missing values BEFORE cleaning:
mpg             0
```

```
cylinders       0
displacement    0
horsepower      0
weight          0
acceleration    0
model_year      0
origin          0
car_name        0
dtype: int64
```

```
# 3.2 Convert horsepower to numeric (coerce non-numeric to NaN)
auto["horsepower"] = pd.to_numeric(auto["horsepower"],
errors="coerce")


# 3.3 Drop rows with missing horsepower
rows_before = auto.shape[0]
auto = auto.dropna(subset=["horsepower"])
rows_after = auto.shape[0]

print(f"\nRows before cleaning: {rows_before}")
print(f"Rows after dropping missing horsepower: {rows_after}")
print(f"Rows removed: {rows_before - rows_after}")
```

```
Rows before cleaning: 398
Rows after dropping missing horsepower: 392
Rows removed: 6
```

```
# 3.4 Ensure correct dtypes
auto["horsepower"] = auto["horsepower"].astype("float64")
auto["weight"] = auto["weight"].astype("int64")

print("\nDtypes AFTER cleaning:")
print(auto.dtypes)

print("\nMissing values AFTER cleaning:")
print(auto.isna().sum())
```

```
Dtypes AFTER cleaning:
mpg             float64
cylinders         int64
displacement    float64
horsepower      float64
weight            int64
acceleration    float64
model_year        int64
origin            int64
car_name         object
```

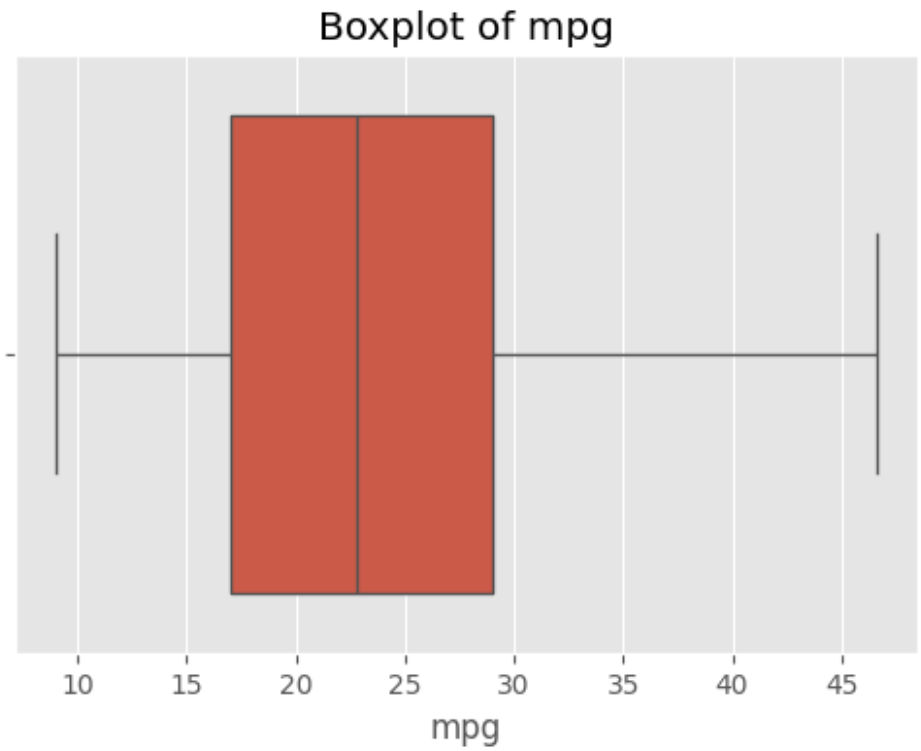
```
dtype: object

Missing values AFTER cleaning:
mpg             0
cylinders       0
displacement    0
horsepower      0
weight          0
acceleration    0
model_year      0
origin          0
car_name        0
dtype: int64
```

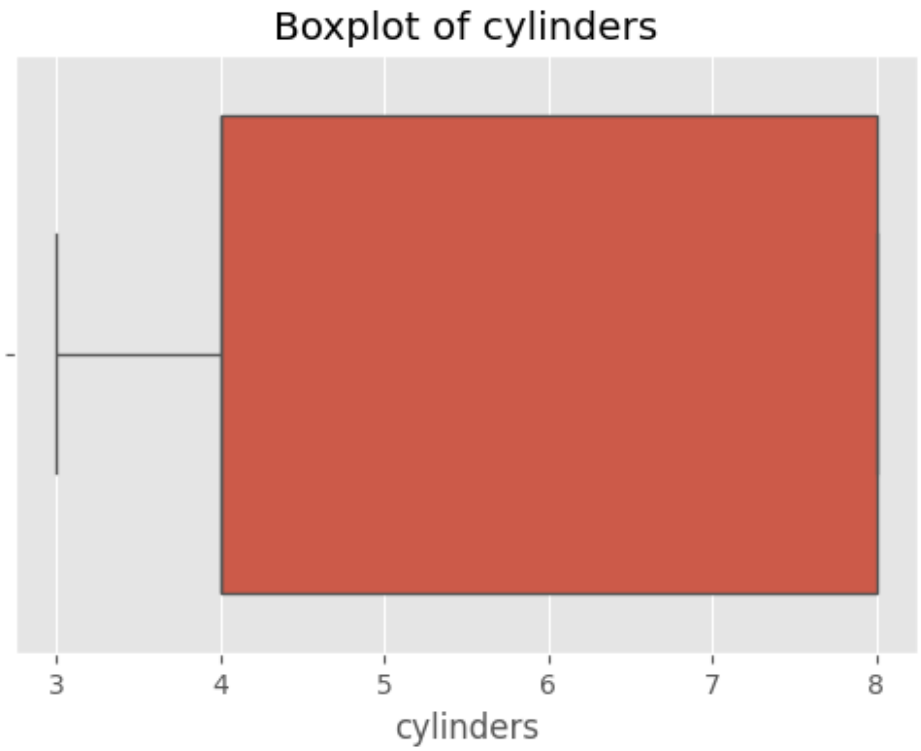
In the cleaning step I focused on correcting data types and handling missing values, which is essential to avoid errors and biased models later. First, I converted the horsepower column from object to numeric using pd.to_numeric(..., errors="coerce"), which turns non-numeric entries (for example "?") into NaN. After this conversion I dropped rows with missing horsepower, reducing the dataset from 398 to 392 observations; this ensures all remaining rows have a complete set of predictor values. I then explicitly cast horsepower to float64 and weight to int64 so that all feature columns used in modelling are in appropriate numeric formats. The final dtypes and missing-value checks confirm that there are no remaining NaN values in the cleaned dataset, meaning the data is ready for exploratory analysis and model training without additional imputation.

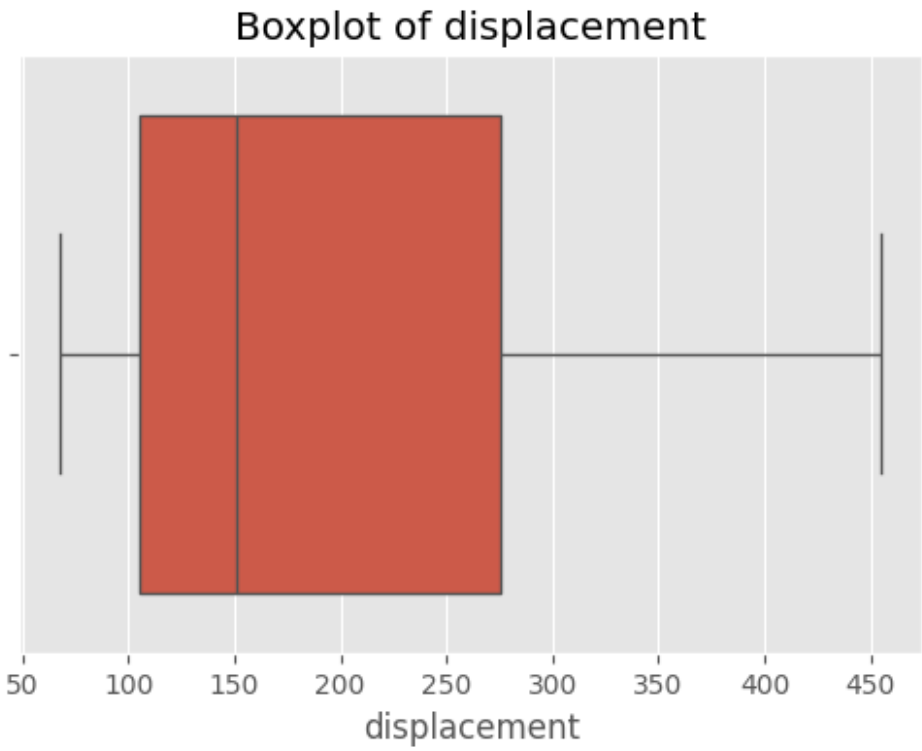
4. Exploratory Data Analysis

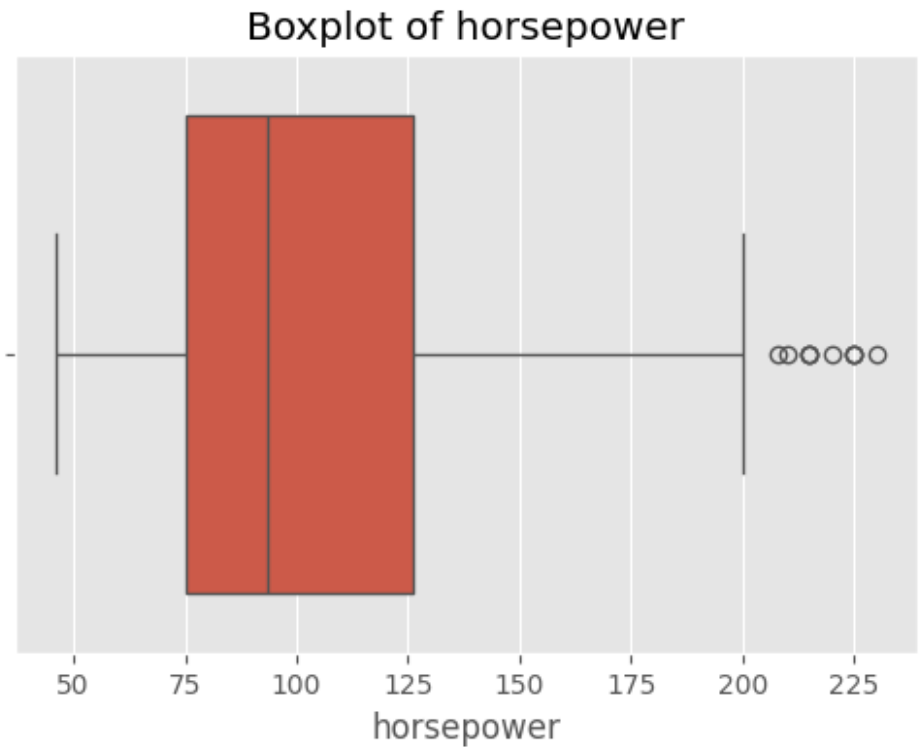
4.1 – Boxplots for outliers

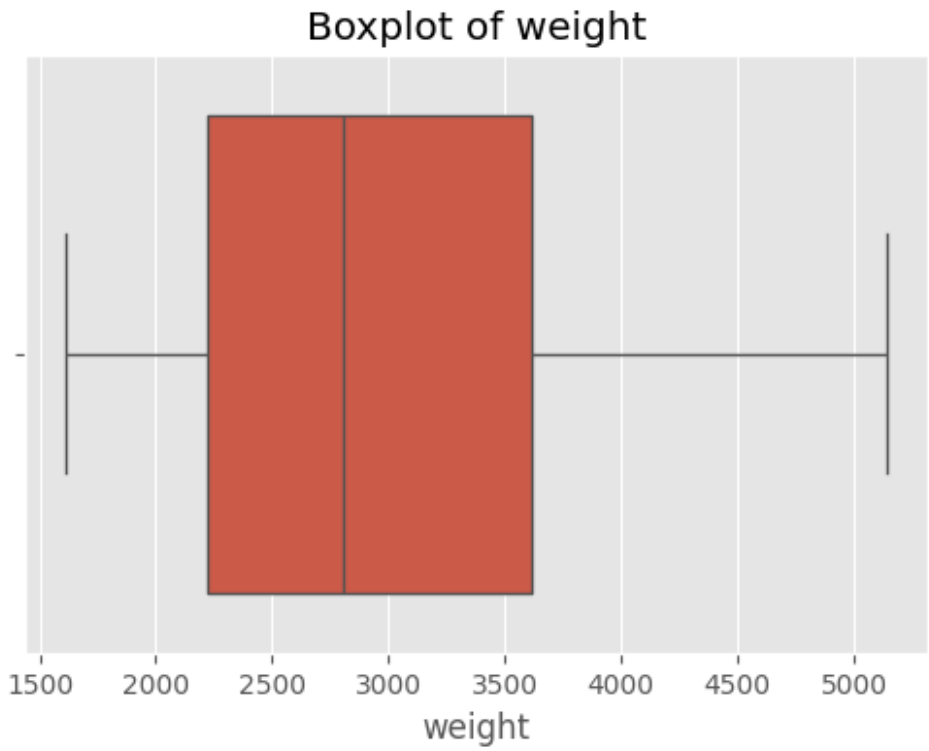
```python
# ========================================================
# 4. EXPLORATORY DATA ANALYSIS (EDA)
# ========================================================

# 4.1 Detecting Outliers with Boxplots

num_cols = [
    "mpg", "cylinders", "displacement", "horsepower",
    "weight", "acceleration", "model_year", "origin"
]

for col in num_cols:
    plt.figure(figsize=(6, 4))
    sns.boxplot(x=auto[col])
    plt.title(f"Boxplot of {col}")
    plt.xlabel(col)
    plt.show()
```

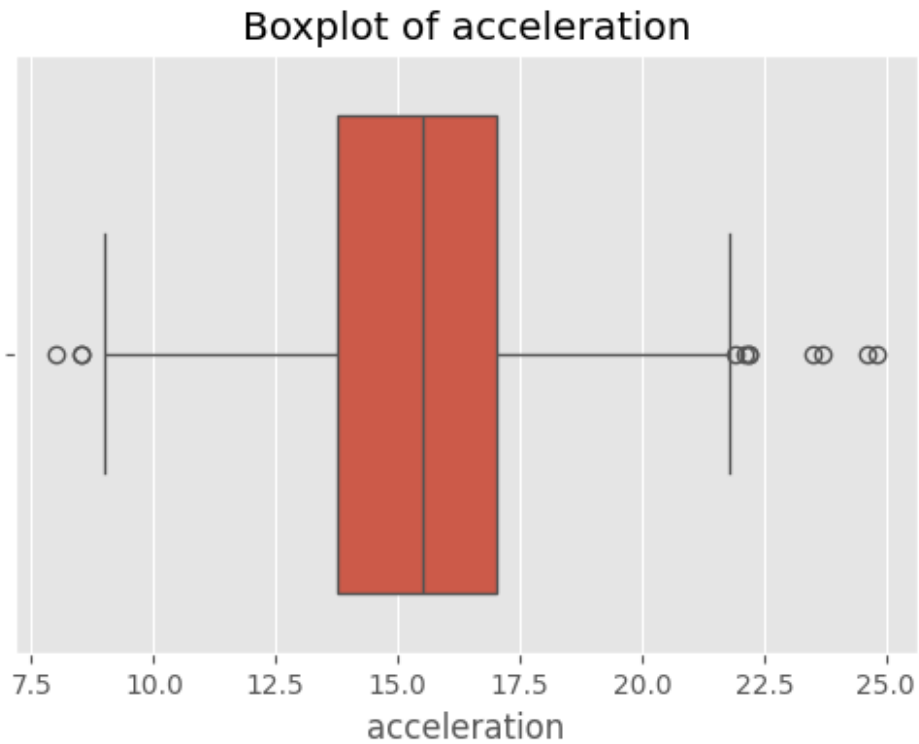

Boxplot of mpg
10
15
20
25
30
35
40
45
mpg

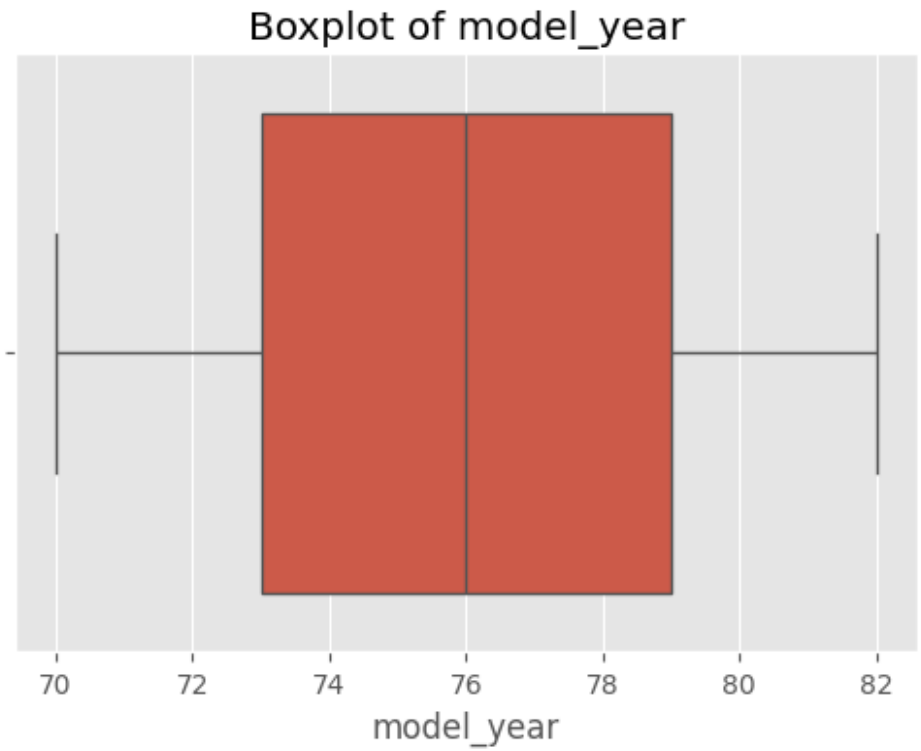

Boxplot of cylinders
3
4
5
6
7
8
cylinders

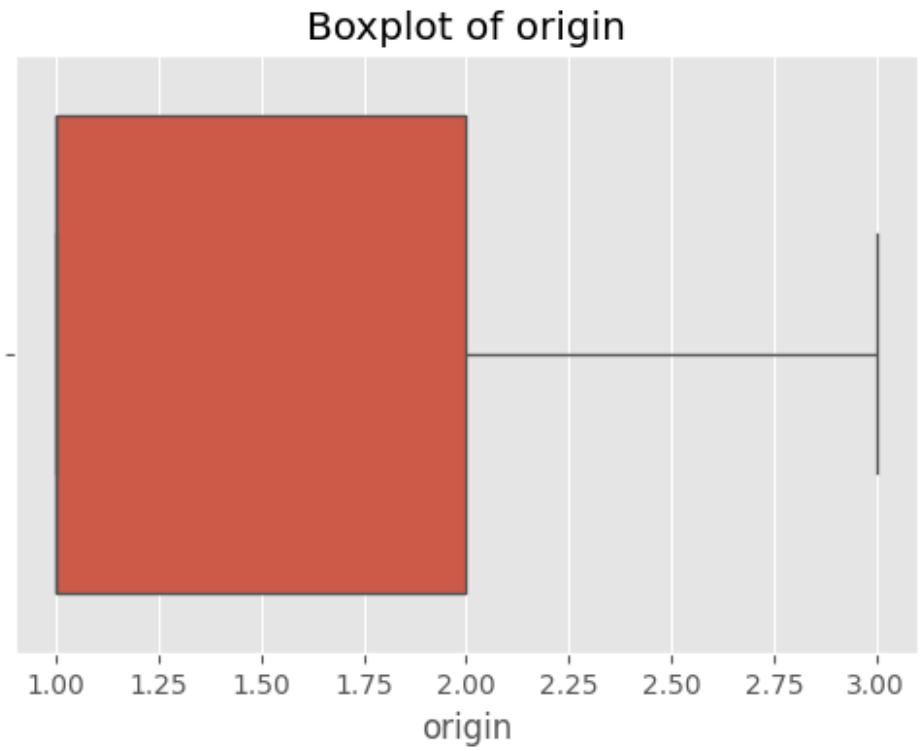

Boxplot of displacement
50
100
150
200
250
300
350
400
450
displacement


Boxplot of horsepower
50
75
100
125
150
175
200
225
horsepower


Boxplot of weight
1500
2000
2500
3000
3500
4000
4500
5000
weight


Boxplot of acceleration
7.5
10.0
12.5
15.0
17.5
20.0
22.5
25.0
acceleration





The boxplots for each numerical variable give a visual summary of central tendency, spread and potential outliers. For mpg, most cars cluster between about 15 and 30 mpg, with a few higher-efficiency vehicles appearing as points above the upper whisker. The displacement and

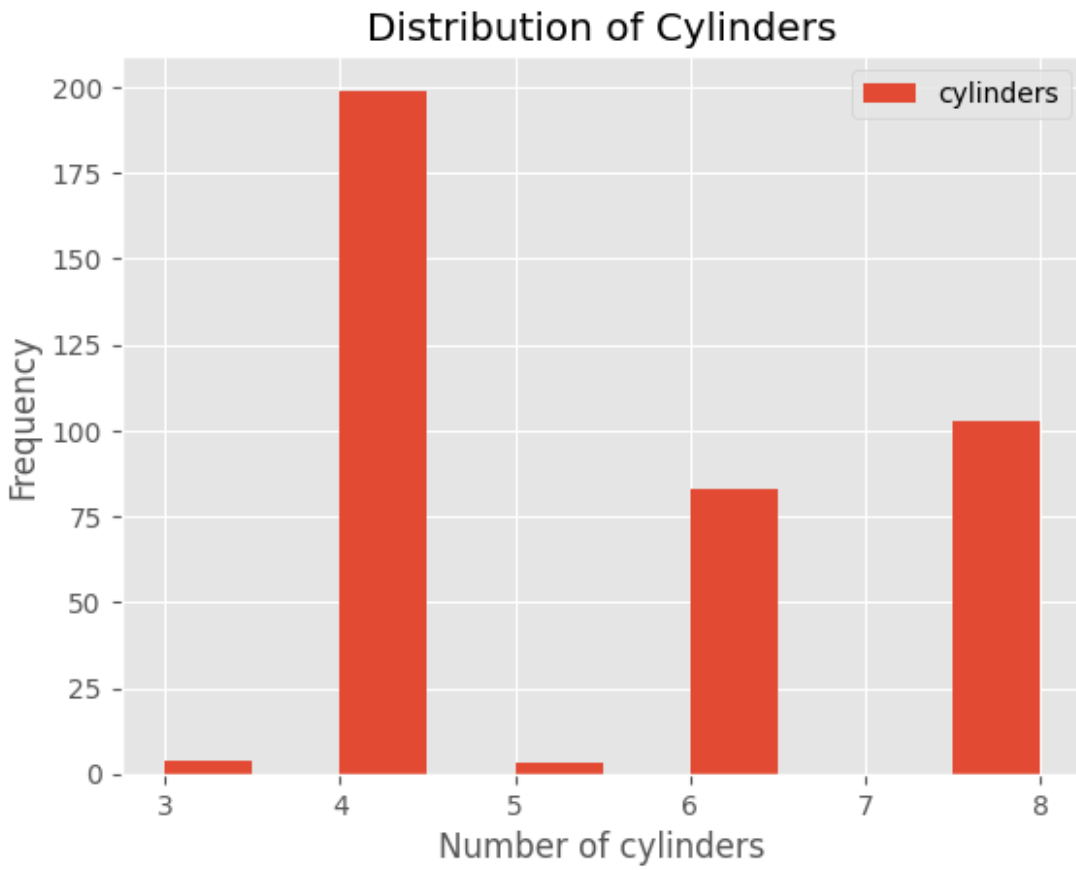
weight boxplots show a long upper tail, indicating some very large engines and heavy cars that could influence model fitting if not handled carefully. Horsepower also has several high-value outliers around 200–230 HP, while acceleration reveals both very slow and very fast cars at the extremes. The boxplots for model_year and origin look more uniform, reflecting the discrete coding of these variables. Overall, this step helps identify where extreme values exist so that I can be aware of their impact on correlations and model performance.

4.2 – Histograms for key discrete variables

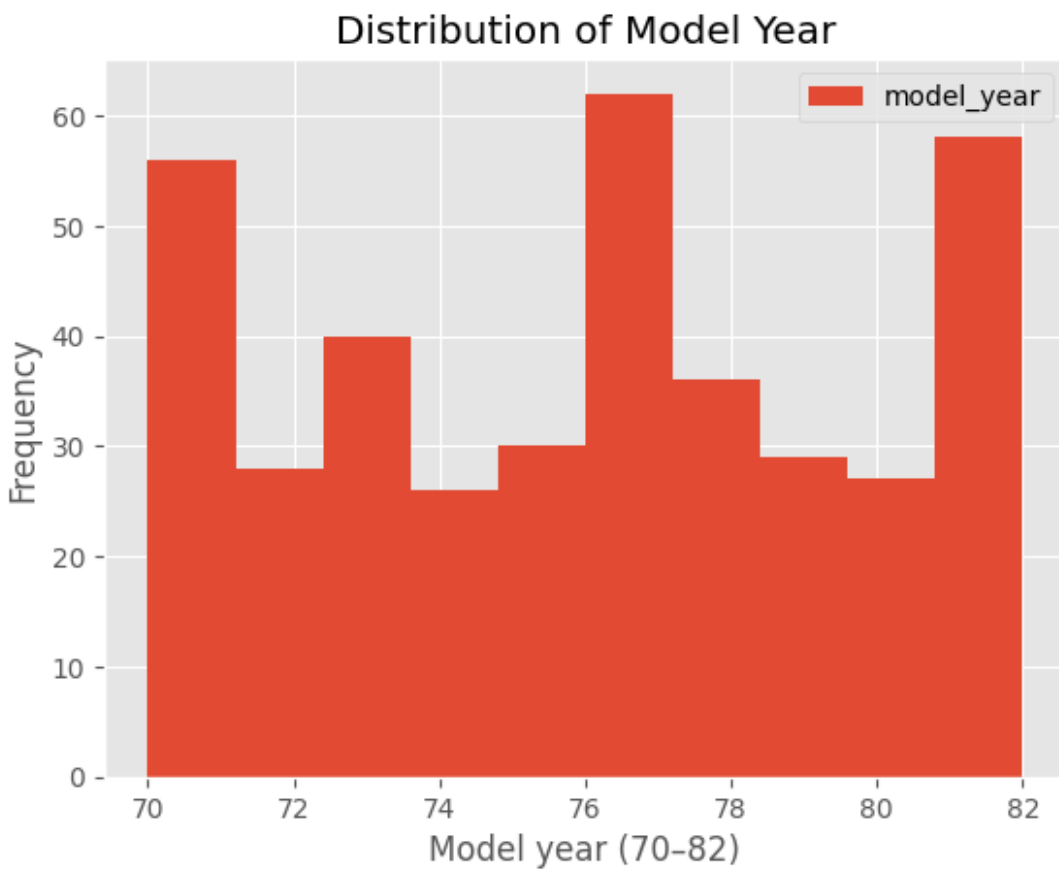
```
# ========================================================
# 4.2 Distributions of Origin, Cylinders and Model Year
# ========================================================

auto[["origin"]].plot.hist()
plt.title("Distribution of Origin")
plt.xlabel("Origin code")
plt.ylabel("Frequency")
plt.show()
```

```
auto[["cylinders"]].plot.hist()
plt.title("Distribution of Cylinders")
```

```
plt.xlabel("Number of cylinders")
plt.ylabel("Frequency")
plt.show()
```



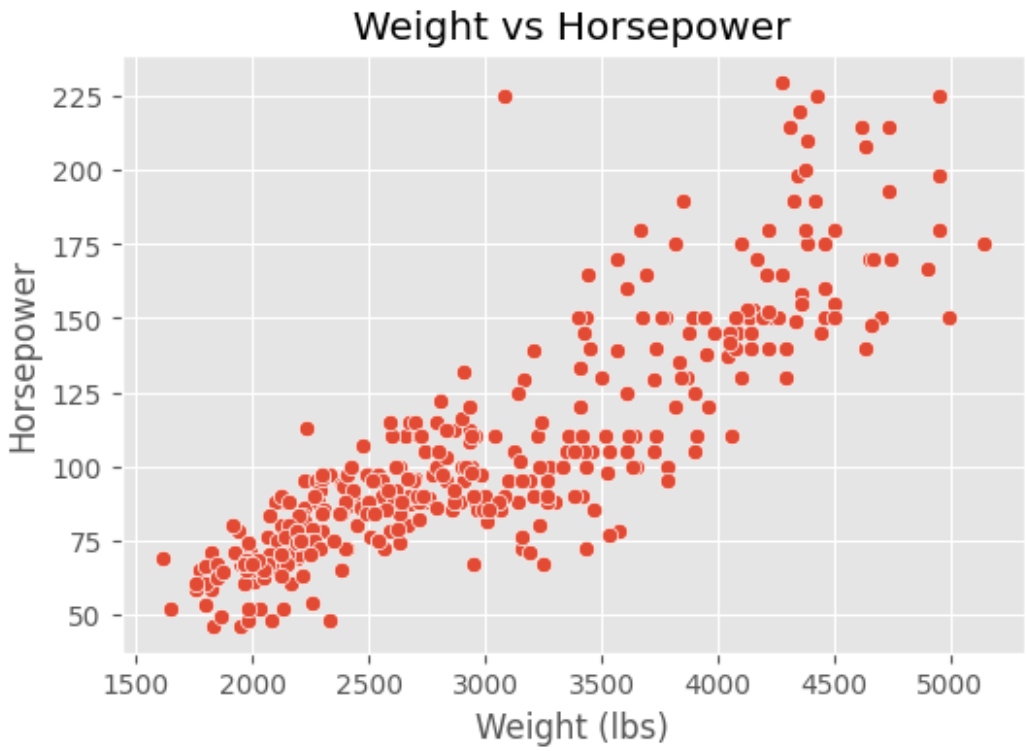

```
auto[["model_year"]].plot.hist()
plt.title("Distribution of Model Year")
plt.xlabel("Model year (70–82)")
plt.ylabel("Frequency")
plt.show()
```



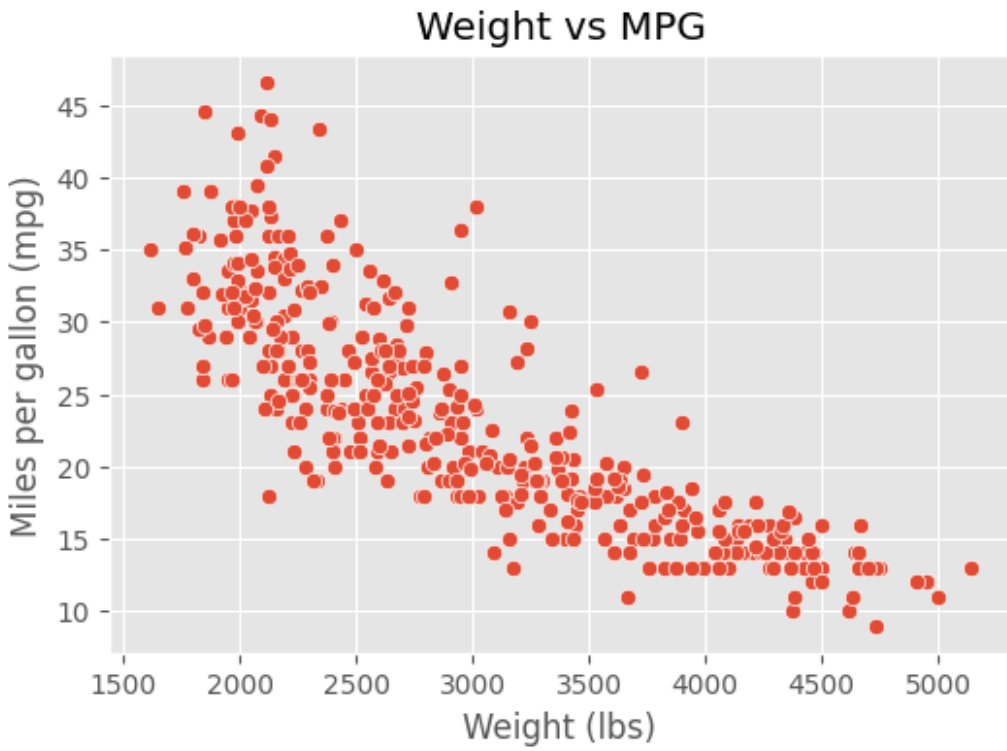
These histograms summarise how the discrete variables are distributed across the dataset. The origin histogram and pie chart show that around two-thirds of vehicles are from origin code 1 (U.S.), with smaller but still substantial proportions from origins 2 and 3, which likely correspond to European and Japanese manufacturers. The cylinders histogram reveals that most vehicles have 4 cylinders, followed by 8-cylinder cars and a smaller group of 6-cylinder engines; this suggests the dataset covers both economy and performance-oriented vehicles. The model_year histogram indicates that cars are fairly spread across years 1970–1982, with some years slightly more represented, which is important because regulatory changes and technology improvements over time affect fuel efficiency. Understanding these distributions helps interpret later patterns, for example why mpg tends to improve for newer model years.

4.3 – Scatter plots for relationships

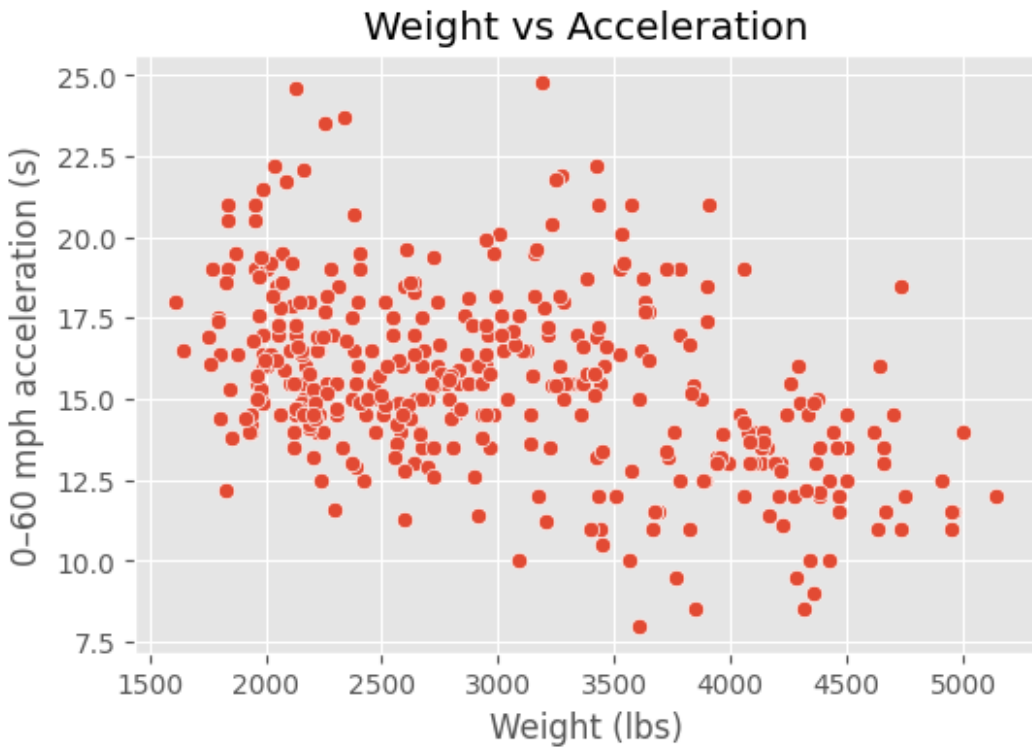
```
# 4.3 Relationships between key numerical variables

plt.figure(figsize=(6, 4))
sns.scatterplot(x="weight", y="horsepower", data=auto)
plt.title("Weight vs Horsepower")
plt.xlabel("Weight (lbs)")
plt.ylabel("Horsepower")
plt.show()
```



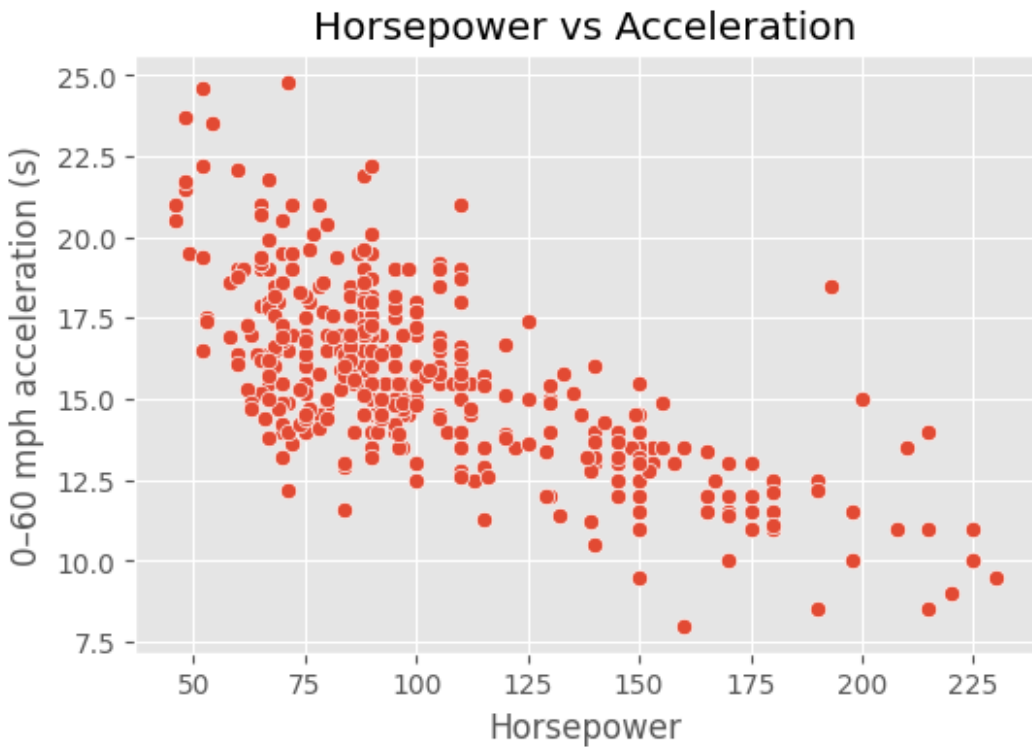

```
plt.figure(figsize=(6, 4))
sns.scatterplot(x="weight", y="mpg", data=auto)
plt.title("Weight vs MPG")
plt.xlabel("Weight (lbs)")
plt.ylabel("Miles per gallon (mpg)")
plt.show()
```



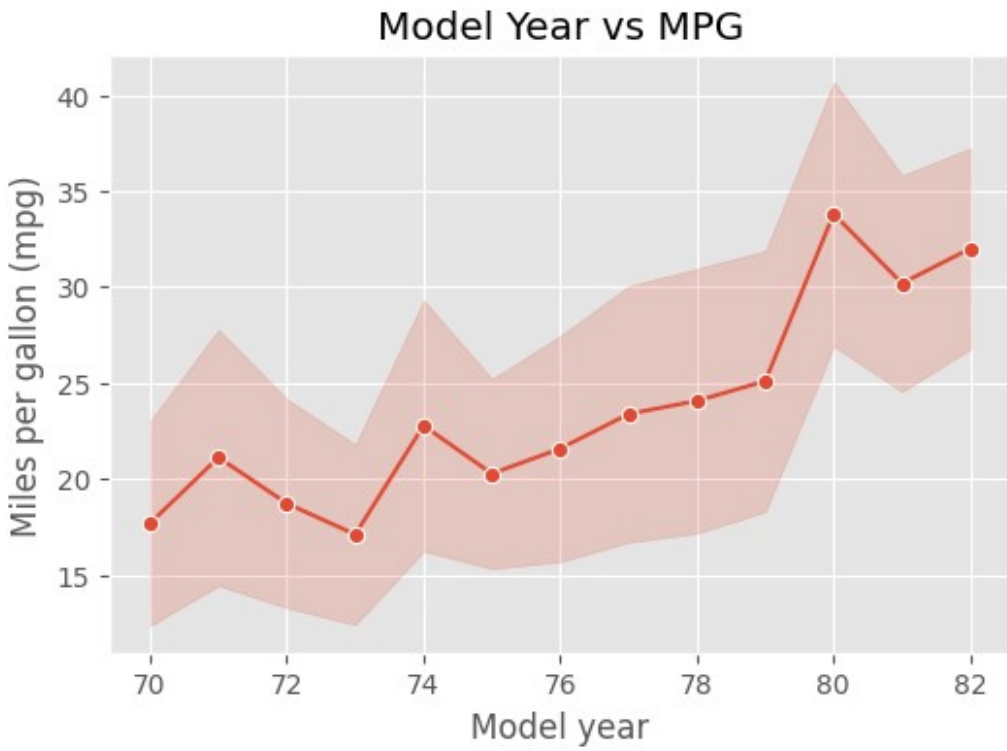

```
plt.figure(figsize=(6, 4))
sns.scatterplot(x="weight", y="acceleration", data=auto)
plt.title("Weight vs Acceleration")
plt.xlabel("Weight (lbs)")
plt.ylabel("0–60 mph acceleration (s)")
plt.show()
```




```
plt.figure(figsize=(6, 4))
sns.scatterplot(x="horsepower", y="acceleration", data=auto)
plt.title("Horsepower vs Acceleration")
plt.xlabel("Horsepower")
plt.ylabel("0–60 mph acceleration (s)")
plt.show()
```




```
plt.figure(figsize=(6, 4))
sns.lineplot(x="model_year", y="mpg", data=auto, ci="sd", marker="o")
plt.title("Model Year vs MPG")
plt.xlabel("Model year")
plt.ylabel("Miles per gallon (mpg)")
plt.show()
```

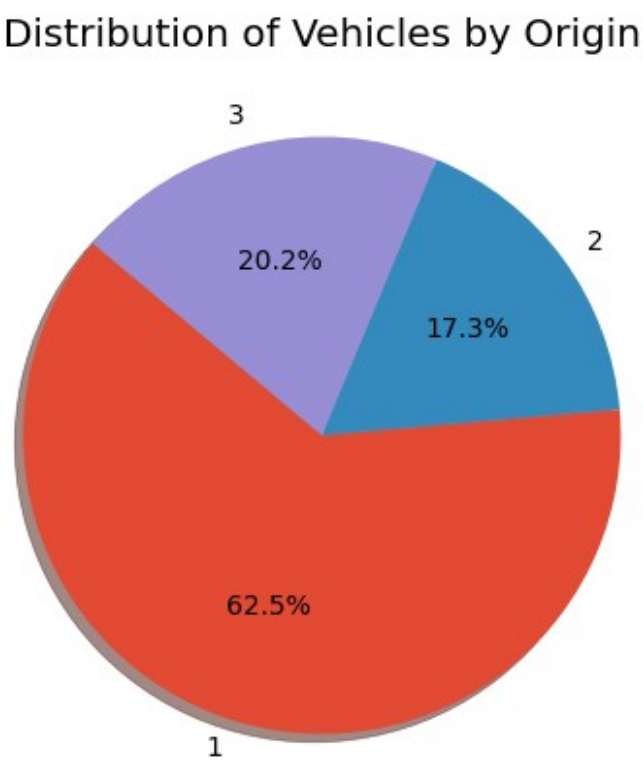


The scatter plots explore pairwise relationships between key variables and give an intuitive sense of which features are likely to be most predictive of mpg. The weight vs horsepower plot shows a strong positive trend: heavier cars tend to have more powerful engines, which is consistent with engineering expectations. The weight vs mpg plot shows a clear negative relationship, where heavier cars generally have lower fuel efficiency; this suggests weight will be a very important predictor. The weight vs acceleration and horsepower vs acceleration plots show that more powerful or heavier vehicles often have lower 0–60 mph times, although there is some spread due to design differences. The line plot of model_year vs mpg displays an upward trend in average mpg over time, reflecting improvements in engine technology and the impact of fuel economy regulations. Together, these plots justify including variables such as weight, displacement, horsepower and model_year in the regression models.

4.4 – Origin distribution pie chart

```
# 4.4 Origin distribution (pie chart)

sizes = auto["origin"].value_counts().sort_index()

plt.figure(figsize=(5, 5))
plt.pie(
    sizes.values,
    labels=sizes.index,
    autopct="%1.1f%%",
    shadow=True,
    startangle=140
)
```

```
plt.title("Distribution of Vehicles by Origin")
plt.show()

sizes
```

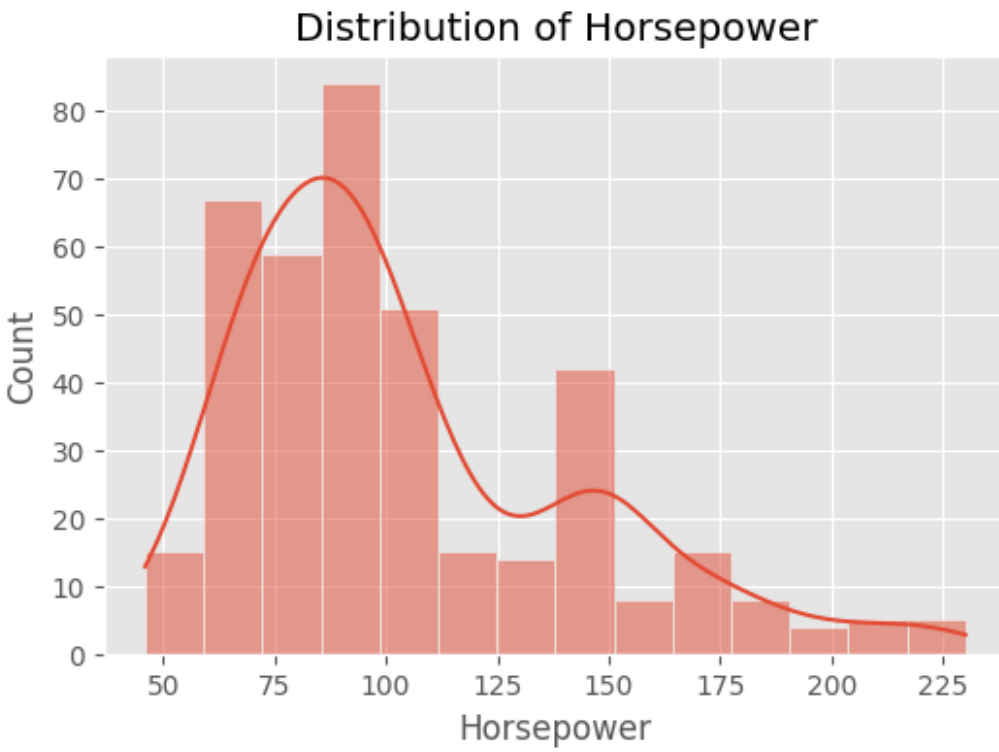



```
origin
1    245
2     68
3     79
Name: count, dtype: int64
```

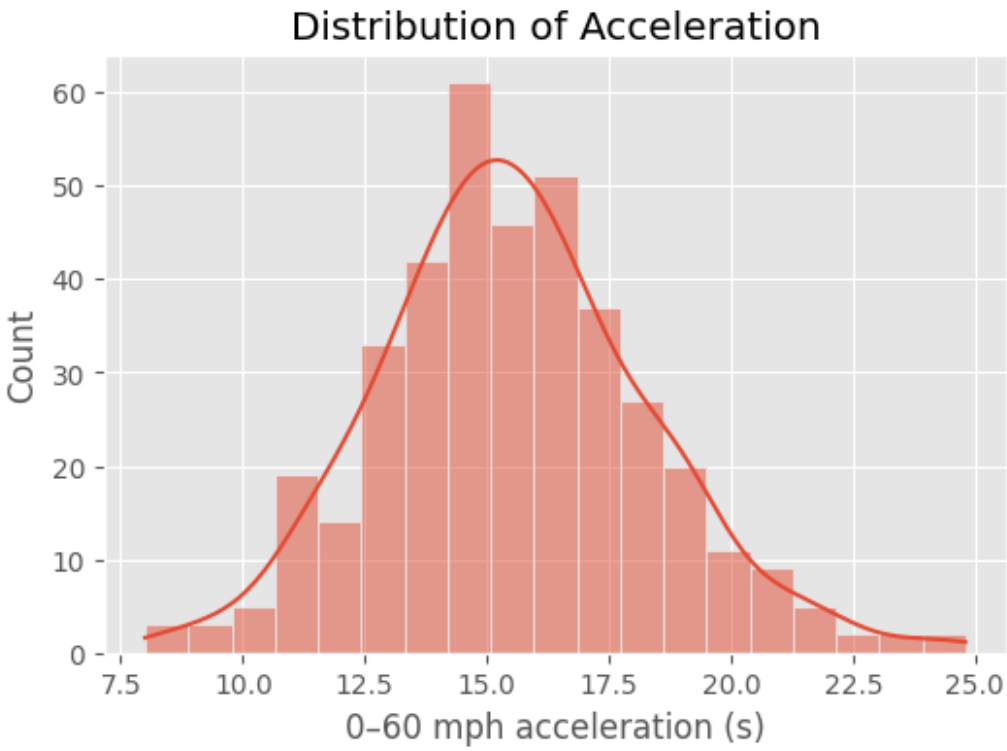
The origin pie chart quantifies the proportion of vehicles from each manufacturing region. Approximately 62.5% of the cars are from origin 1, around 17.3% from origin 2, and about 20.2% from origin 3, so the dataset is dominated by one origin but still contains a meaningful number of vehicles from other regions. This matters because origin acts as a proxy for differences in design philosophy and regulations: for example, Japanese and European cars in this period often emphasised higher fuel efficiency than many American models. Including origin as a categorical predictor therefore allows the models to capture systematic differences between regions that are not fully explained by engine size or weight alone.

4.5 Numerical Distributions (KDE Histograms)

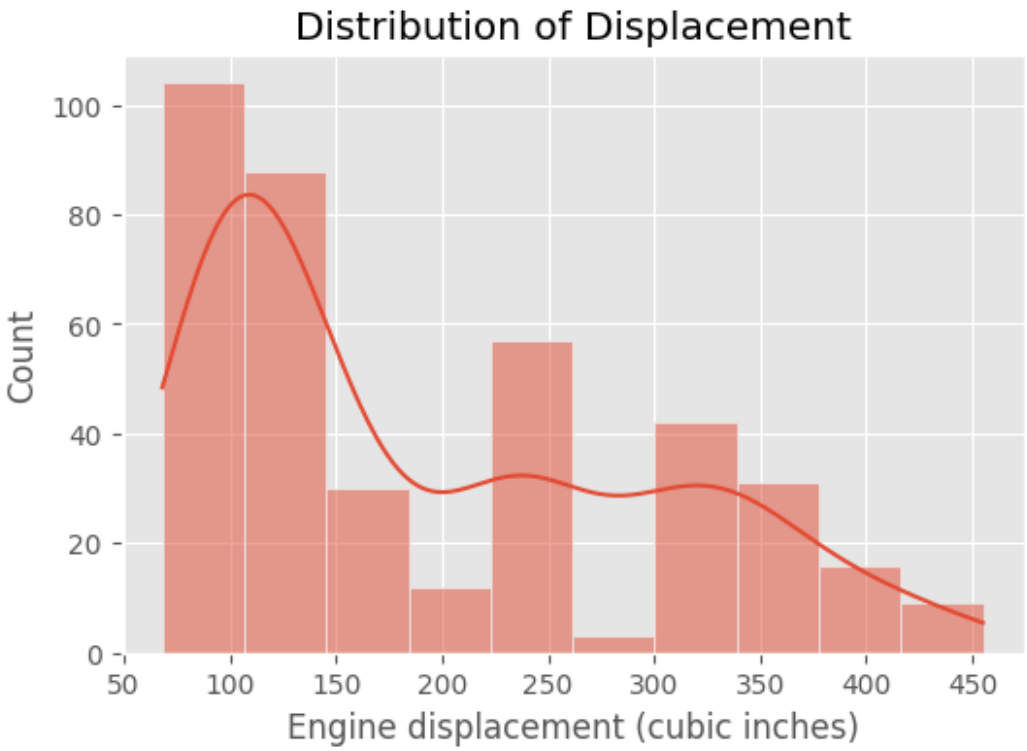
```
# ========================================================
# 4.5 Numerical Feature Distributions
# ========================================================

plt.figure(figsize=(6, 4))
sns.histplot(x="horsepower", data=auto, kde=True)
plt.title("Distribution of Horsepower")
plt.xlabel("Horsepower")
plt.show()
```



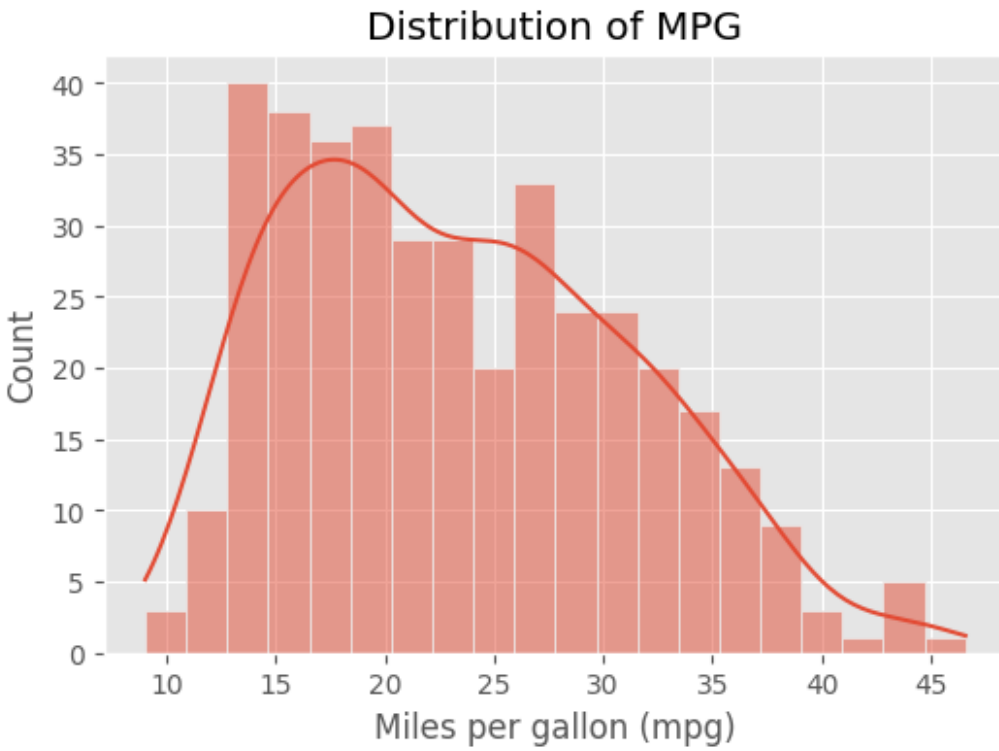

```
plt.figure(figsize=(6, 4))
sns.histplot(x="acceleration", data=auto, kde=True)
plt.title("Distribution of Acceleration")
plt.xlabel("0–60 mph acceleration (s)")
plt.show()
```




```
plt.figure(figsize=(6, 4))
sns.histplot(x="displacement", data=auto, kde=True)
plt.title("Distribution of Displacement")
plt.xlabel("Engine displacement (cubic inches)")
plt.show()
```




```
plt.figure(figsize=(6, 4))
sns.histplot(x="mpg", data=auto, kde=True, bins=20)
plt.title("Distribution of MPG")
plt.xlabel("Miles per gallon (mpg)")
plt.ylabel("Count")
plt.show()
```



The KDE-enhanced histograms help to understand the shapes of the main numerical distributions. Horsepower is right-skewed, with many cars clustered between about 60 and 120 HP and relatively few very high-powered vehicles; this suggests that a simple linear transformation may be adequate without needing log transformations. Acceleration appears roughly bell-shaped, centred around 15 seconds, which is favourable for models that assume approximately normal errors. Displacement and weight are both skewed towards larger values, mirroring the presence of big-engine, heavy cars in the dataset. The mpg distribution is also slightly right-skewed, with most vehicles between 15 and 30 mpg and a tail of more efficient cars above 35 mpg. Overall, these patterns indicate that some relationships may be non-linear but still manageable with tree-based models or regularised linear models.

4.B1 – Correlation matrix

```
# Ensure auto1 is numeric-only (we already dropped car_name)
auto1 = auto.drop("car_name", axis=1)

# Correlation matrix
corr_matrix = auto1.corr()
corr_matrix
```

{"summary":"{\n \"name\": \"corr_matrix\",\n \"rows\": 8,\n \"fields\": [\n {\n \"column\": \"mpg\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 0.7872618248401377,\n \"min\": -0.832244214831575,\n \"max\": 1.0,\n \"num_unique_values\": 8,\n \"samples\": [\n -0.7776175081260227,\n 0.42332853690278693,\n 1.0\n],\n \"semantic_type\": \"\",\n

\"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"cylinders\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 0.7967448686550586,\n   \"min\": -0.7776175081260227,\n   \"max\": 1.0,\n   \"num_unique_values\": 8,\n   \"samples\": [\n   1.0,\n   -0.5046833793486444,\n   -0.7776175081260227\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"displacement\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 0.8260065959541416,\n   \"min\": -0.8051269467104577,\n   \"max\": 1.0,\n   \"num_unique_values\": 8,\n   \"samples\": [\n   0.9508233008277828,\n   -0.5438004967345681,\n   -0.8051269467104577\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"horsepower\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 0.8039516732687065,\n   \"min\": -0.7784267838977761,\n   \"max\": 1.0,\n   \"num_unique_values\": 8,\n   \"samples\": [\n   0.8429833569186568,\n   -0.6891955103342376,\n   -0.7784267838977761\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"weight\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 0.7952179376818062,\n   \"min\": -0.832244214831575,\n   \"max\": 1.0,\n   \"num_unique_values\": 8,\n   \"samples\": [\n   0.8975273403408176,\n   -0.4168392020037062,\n   -0.832244214831575\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"acceleration\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 0.5977834814184086,\n   \"min\": -0.6891955103342376,\n   \"max\": 1.0,\n   \"num_unique_values\": 8,\n   \"samples\": [\n   -0.5046833793486444,\n   1.0,\n   0.42332853690278693\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"model_year\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 0.5254541182328929,\n   \"min\": -0.41636147709998894,\n   \"max\": 1.0,\n   \"num_unique_values\": 8,\n   \"samples\": [\n   -0.34564744033808426,\n   0.29031611333651997,\n   0.5805409660907859\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"origin\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 0.6140847152750115,\n   \"min\": -0.6145351145726985,\n   \"max\": 1.0,\n   \"num_unique_values\": 8,\n   \"samples\": [\n   -0.5689315895422126,\n   0.21274580799458218,\n   0.5652087567164604\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   }\n]\n}","type":"dataframe","variable_name":"corr_matrix"}

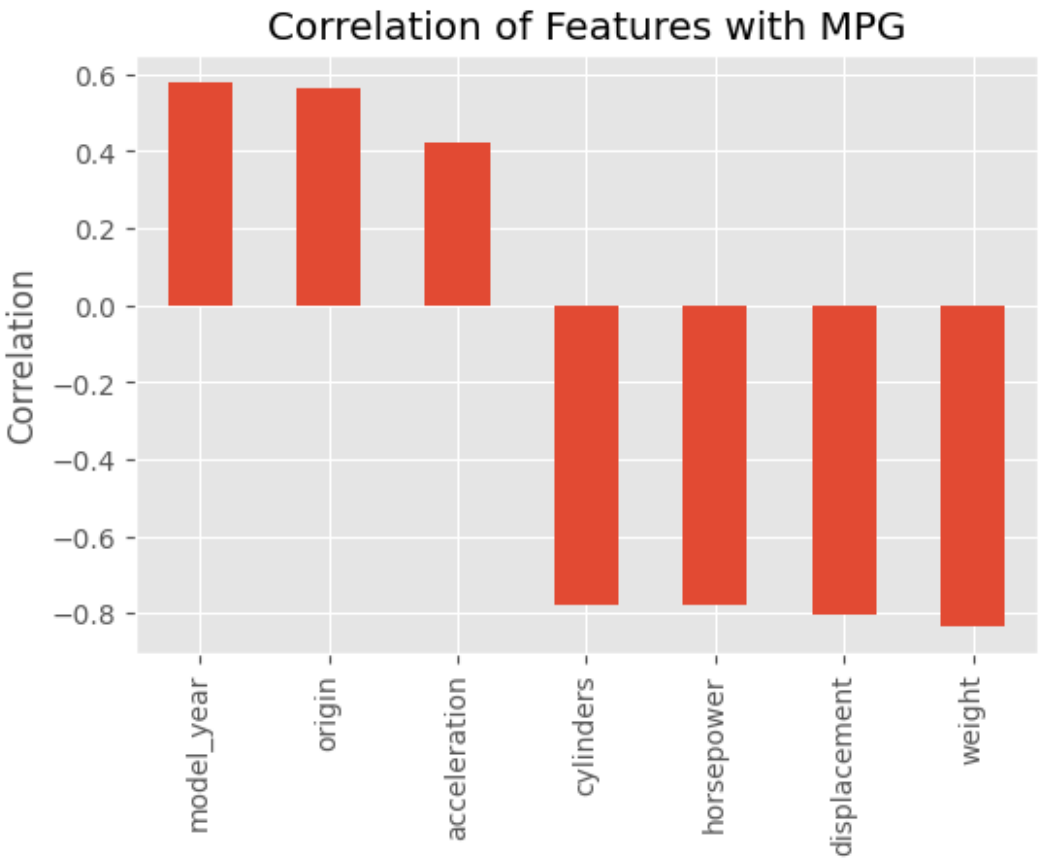
```
import plotly.express as px
```

```
fig = px.imshow(
    corr_matrix,
    color_continuous_scale="RdBu",
    text_auto=True,
    labels=dict(color="Correlation")
)
fig.update_layout(
    width=900,
    height=900,
    title="Correlation Heatmap of Numerical Features"
)
fig.show()
```

The correlation matrix shows how strongly each pair of numerical variables is linearly related. mpg has strong negative correlations with cylinders, displacement, horsepower and especially weight, confirming that larger, heavier and more powerful cars are less fuel-efficient. In contrast, mpg is positively correlated with model_year and origin, suggesting that newer cars and those from certain regions tend to achieve better fuel economy. The strong positive correlations among cylinders, displacement, horsepower and weight indicate multicollinearity: these engine-size variables are measuring similar underlying concepts. This insight is important for linear models, where multicollinearity can make coefficient estimates unstable, but it is less problematic for tree-based methods like Random Forests.

4.B2 - Correlation with mpg only (easier story + feature choice).

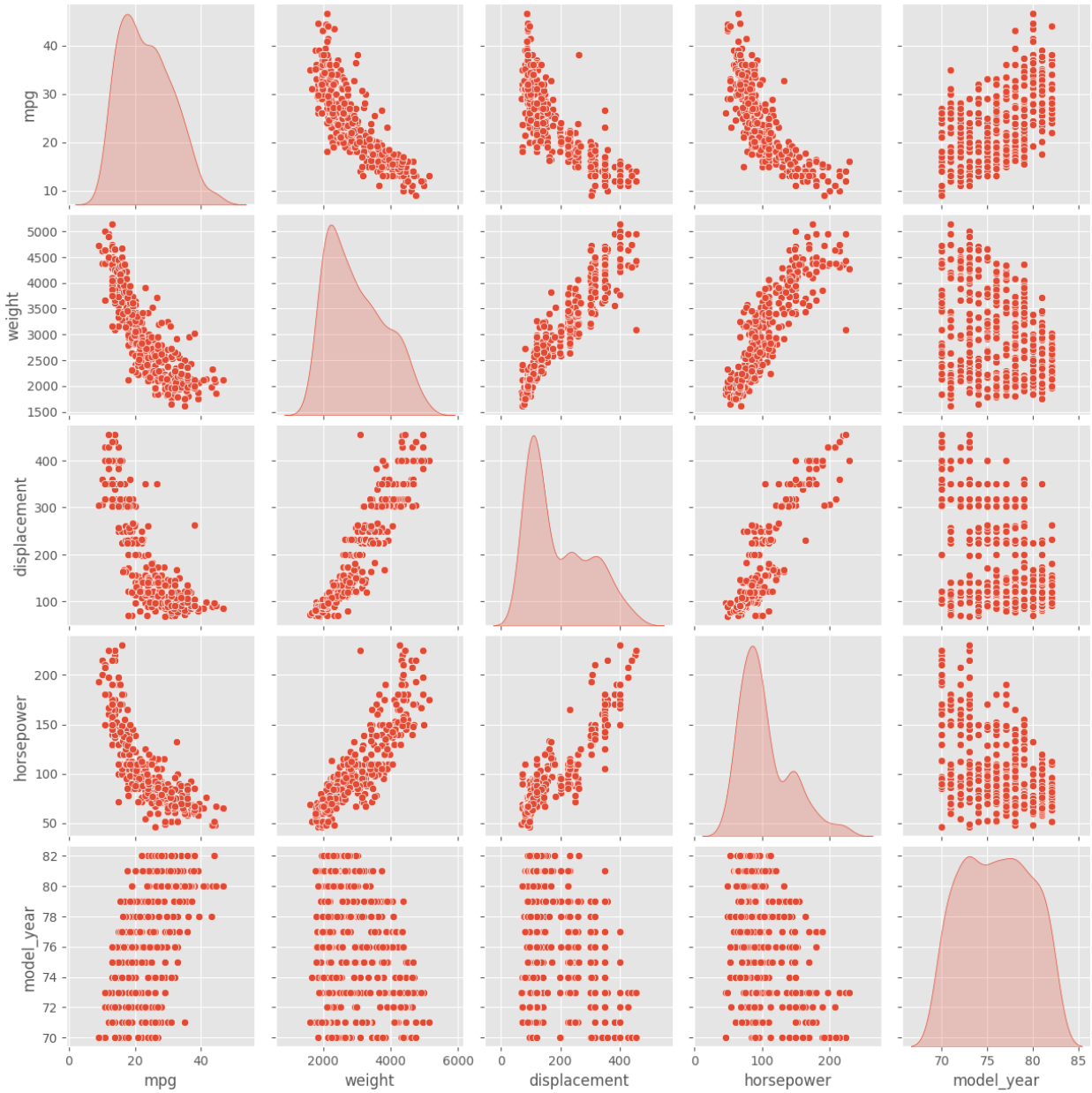
```
mpg_corr = auto1.corr()["mpg"].sort_values(ascending=False)
mpg_corr
mpg_corr.drop("mpg").plot(kind="bar", figsize=(6,4))
plt.title("Correlation of Features with MPG")
plt.ylabel("Correlation")
plt.show()
```



Plotting the correlation of each feature with mpg in a bar chart provides a simple ranking of which variables are most strongly associated with fuel efficiency. Weight shows the largest negative correlation with mpg, followed by displacement and horsepower, reinforcing the idea that overall vehicle size and engine capacity are key drivers of fuel consumption. On the positive side, model_year and origin have moderate correlations with mpg, reflecting efficiency improvements over time and regional design differences. This ranking is useful for feature selection because it highlights which predictors are most promising for inclusion in simpler models, and it also helps interpret feature importance in later Random Forest results.

4.82- Simple pairplot or scatter vs mpg.

```
sns.pairplot(
    auto1[["mpg", "weight", "displacement", "horsepower",
"model_year"]],
    diag_kind="kde"
)
plt.show()
```



The pairplot offers a compact view of both marginal distributions and pairwise scatter plots for the most important variables. The diagonal plots confirm earlier observations about distribution shapes, while the off-diagonal panels show clear non-linear trends: for example, mpg decreases in a curved pattern as weight, displacement and horsepower increase. The relationship between model_year and mpg again shows that newer models tend to be more fuel efficient. These scatter patterns suggest that purely linear models may not capture all the structure in the data, which motivates trying more flexible techniques such as Decision Trees and Random Forests alongside linear and regularised regressions.

5. Feature Selection and Train–Test Split

5.1 – Choose features and split data

```
# ========================================================
# 5. FEATURE SELECTION AND TRAIN–TEST SPLIT
# ========================================================

# Option A: full numeric feature set (keep model_year and origin)
features_full = ["cylinders", "displacement", "horsepower",
                 "weight", "acceleration", "model_year", "origin"]


target = "mpg"

# For now use full feature set for modelling
X = auto[features_full]
y = auto[target]

# Train–test split (80/20)
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=21
)

X_train.shape, X_test.shape, y_train.shape, y_test.shape
```

```
((313, 7), (79, 7), (313,), (79,))
```

```
# Option B: reduced feature set (drop model_year and origin)

features_reduced = ["cylinders", "displacement", "horsepower",
                    "weight", "acceleration"]
```

In this section I defined the feature matrix and target variable, then split the data into training and test sets. The full feature set includes cylinders, displacement, horsepower, weight, acceleration, model_year and origin, with mpg as the target. Using train_test_split with an 80/20 split and a fixed random seed ensures reproducible experiments and allows me to evaluate models on unseen data. I also defined a reduced feature set that excludes model_year and origin, which I later use as a sensitivity check to see how much performance depends on time and region information. This structure follows good practice in supervised learning: fit models on the training data, use cross-validation for selection, and reserve the test set for a final unbiased performance estimate.

6. Baseline Linear Regression Model

6.1 Code – Train and evaluate Linear Regression with CV

```
# ========================================================
# 6. BASELINE MODEL – LINEAR REGRESSION
# ========================================================

baseline_model = LinearRegression()
```

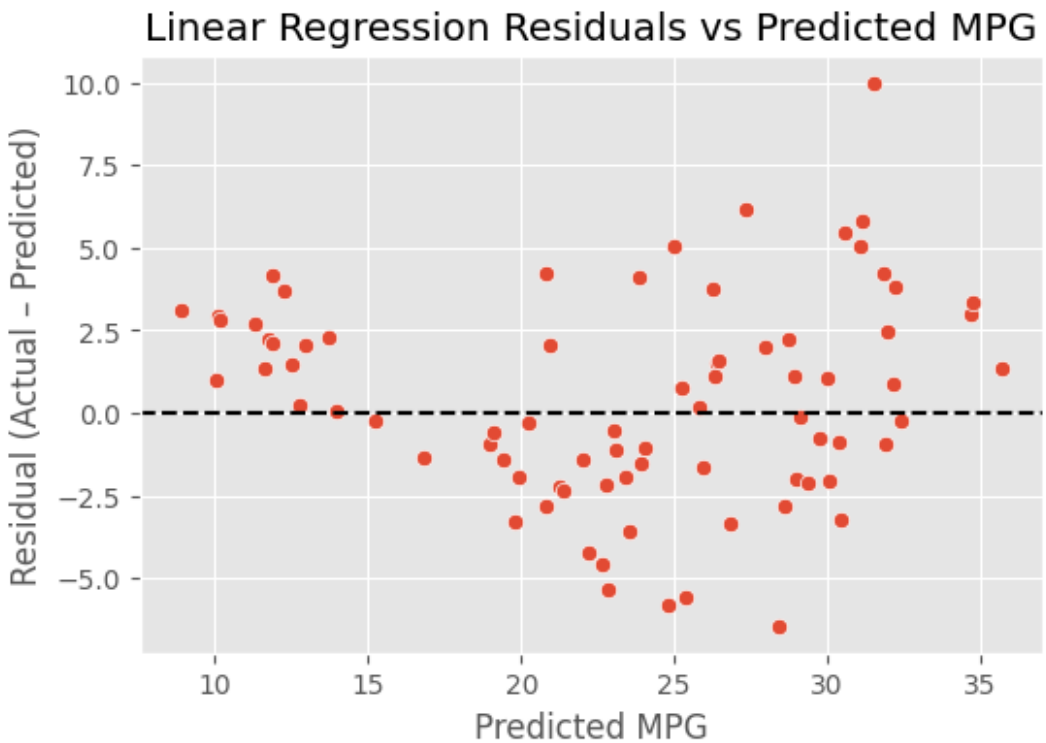
```
# 5-fold cross-validation on training set
cv_rmse = cross_val_score(
    baseline_model, X_train, y_train,
    cv=5,
    scoring="neg_root_mean_squared_error"
)
cv_r2 = cross_val_score(
    baseline_model, X_train, y_train,
    cv=5,
    scoring="r2"
)

print("Linear Regression — fold RMSE:", -cv_rmse)
print("Linear Regression — fold R²:  ", cv_r2)
print("Average RMSE:", -cv_rmse.mean())
print("Average R²:", cv_r2.mean())

# Fit on full training data and evaluate on test set
baseline_model.fit(X_train, y_train)
y_pred_test = baseline_model.predict(X_test)

test_mse = mean_squared_error(y_test, y_pred_test)
test_rmse = np.sqrt(test_mse)
test_r2 = r2_score(y_test, y_pred_test)

print("\nTest MSE:", test_mse)
print("Test RMSE:", test_rmse)
print("Test R²:", test_r2)
```

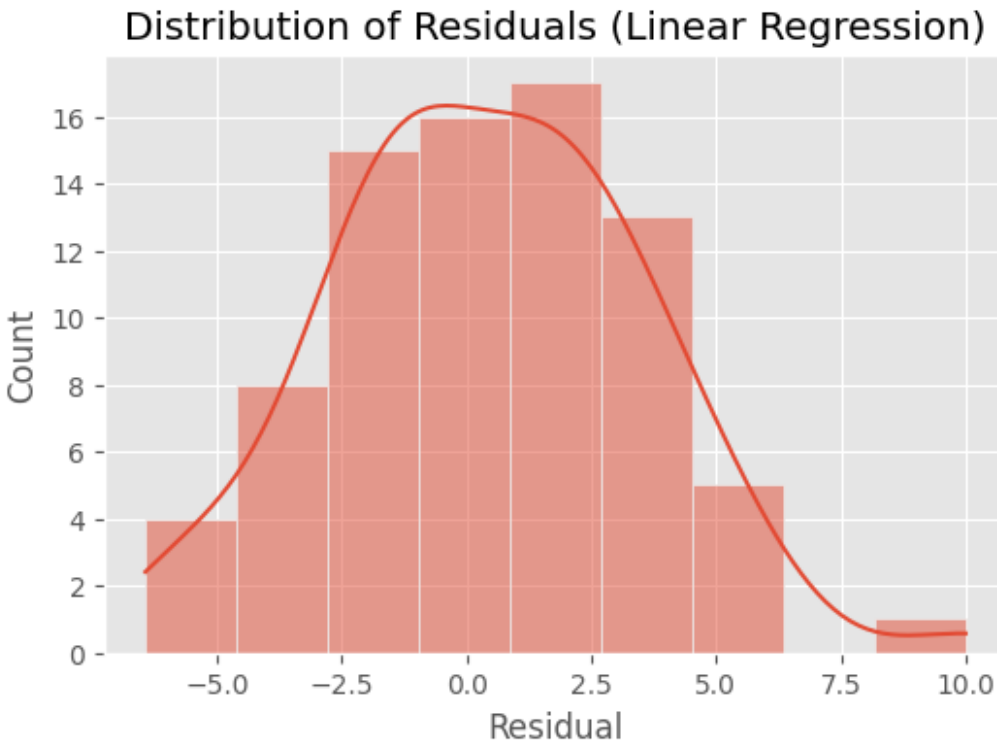
```
Linear Regression — fold RMSE: [3.25282444 3.30266335 2.98698626
4.05154028 3.86781656]
Linear Regression — fold R²:   [0.81435259 0.81074947 0.82497008
0.77933169 0.74808478]
Average RMSE: 3.492366176140289
Average R²: 0.7954977225864415

Test MSE: 9.627592864881494
Test RMSE: 3.10283626137144
Test R²: 0.8374898155078438
```

Check residuals for the Linear Regression baseline.

```
# Predicted vs actual on test set
y_pred_test = baseline_model.predict(X_test)

residuals = y_test - y_pred_test

plt.figure(figsize=(6,4))
sns.scatterplot(x=y_pred_test, y=residuals)
```

```
plt.axhline(0, color="black", linestyle="--")
plt.xlabel("Predicted MPG")
plt.ylabel("Residual (Actual – Predicted)")
plt.title("Linear Regression Residuals vs Predicted MPG")
plt.show()

# Residual distribution
plt.figure(figsize=(6,4))
sns.histplot(residuals, kde=True)
plt.title("Distribution of Residuals (Linear Regression)")
plt.xlabel("Residual")
plt.show()
```





Here I trained a standard Linear Regression model as a baseline and evaluated it using 5-fold cross-validation on the training set. The average cross-validated RMSE is about 3.49 mpg and the average R^2 is roughly 0.80, indicating that a simple linear model already explains around 80% of the variance in mpg. Evaluating on the hold-out test set gives an RMSE of about 3.10 mpg and an R^2 of approximately 0.84, which confirms that the baseline model generalises reasonably well and does not appear heavily over-fitted. The residual vs predicted plot shows residuals scattered around zero with no strong pattern, although there may be slightly larger errors for extreme mpg values. The residual histogram looks approximately symmetric, suggesting that the linear regression assumptions are acceptable but leaving room for improvement with more flexible models.

the reduced feature set as a quick sensitivity check.

```
# Reduced feature set (no model_year, origin)
X_red = auto[features_reduced]

X_train_red, X_test_red, y_train_red, y_test_red = train_test_split(
    X_red, y,
    test_size=0.2,
    random_state=21
)

lin_red = LinearRegression()
cv_rmse_red = cross_val_score(
    lin_red, X_train_red, y_train_red,
    cv=5,
```

```
    scoring="neg_root_mean_squared_error"
)
cv_r2_red = cross_val_score(
    lin_red, X_train_red, y_train_red,
    cv=5,
    scoring="r2"
)

print("Reduced Linear — Avg RMSE:", -cv_rmse_red.mean())
print("Reduced Linear — Avg R²:", cv_r2_red.mean())
```

```
Reduced Linear — Avg RMSE: 4.370397081006307
Reduced Linear — Avg R²: 0.6798170105549709
```

To test the importance of model_year and origin, I refit the Linear Regression model using only cylinders, displacement, horsepower, weight and acceleration. The average cross-validated RMSE increases to about 4.37 mpg and R^2 drops to around 0.68, which is a noticeable deterioration compared with the full feature set. This indicates that model_year and origin add meaningful predictive power beyond the mechanical specifications, likely because they capture improvements in technology and regional design practices. This experiment supports keeping the full set of features for the main modelling work.

1. Baseline Model Comparison with Cross-Validation

7.1 – Define models and simple hyperparameter grids

```
# ========================================================
# 7. BASELINE AND REGULARISED MODELS + TREE MODELS
# ========================================================

from sklearn.model_selection import GridSearchCV

# 1) Linear Regression (already fitted as baseline_model)
linreg = LinearRegression()

# 2) Ridge Regression — small grid over alpha
ridge = Ridge()
ridge_params = {"alpha": [0.01, 0.1, 1.0, 10.0, 100.0]}

# 3) Lasso Regression — small grid over alpha
lasso = Lasso(max_iter=10000)
lasso_params = {"alpha": [0.001, 0.01, 0.1, 1.0, 10.0]}

# 4) Decision Tree Regressor — depth and leaf size
dtr = DecisionTreeRegressor(random_state=21)
dtr_params = {
    "max_depth": [None, 3, 5, 7, 9],
    "min_samples_leaf": [1, 2, 4]
}
```

```python
# 5) Random Forest Regressor — n_estimators and depth
rfr = RandomForestRegressor(random_state=21)
rfr_params = {
    "n_estimators": [100, 300, 500],
    "max_depth": [None, 5, 10],
    "min_samples_leaf": [1, 2, 4]
}
```

7.2 – Helper to run CV + optional GridSearch

```python
def evaluate_model(name, model, param_grid=None):
    """
    If param_grid is provided, runs GridSearchCV (5-fold, RMSE).
    Otherwise, just does 5-fold CV for the base model.
    Returns best_estimator, avg_cv_rmse, avg_cv_r2.
    """
    if param_grid is not None:
        grid = GridSearchCV(
            model,
            param_grid=param_grid,
            cv=5,
            scoring="neg_root_mean_squared_error",
            n_jobs=-1
        )
        grid.fit(X_train, y_train)
        best_model = grid.best_estimator_
        # Get CV scores for best model
        cv_rmse = -grid.best_score_
        # Recompute R² via cross_val_score for clarity
        cv_r2 = cross_val_score(
            best_model, X_train, y_train,
            cv=5,
            scoring="r2"
        ).mean()
    else:
        best_model = model
        cv_rmse_scores = cross_val_score(
            best_model, X_train, y_train,
            cv=5,
            scoring="neg_root_mean_squared_error"
        )
        cv_r2_scores = cross_val_score(
            best_model, X_train, y_train,
            cv=5,
            scoring="r2"
        )
        cv_rmse = -cv_rmse_scores.mean()
        cv_r2 = cv_r2_scores.mean()
        best_model.fit(X_train, y_train)
```

```
    print(f"{name} — Avg CV RMSE: {cv_rmse:.3f}, Avg CV R²:
{cv_r2:.3f}")
    return best_model, cv_rmse, cv_r2
```

7.3 – Run all models

```
results = {}

# Linear Regression (no hyperparameters)
lin_best, lin_rmse, lin_r2 = evaluate_model("LinearRegression",
linreg)
results["LinearRegression"] = (lin_rmse, lin_r2)

# Ridge
ridge_best, ridge_rmse, ridge_r2 = evaluate_model("Ridge", ridge,
ridge_params)
results["Ridge"] = (ridge_rmse, ridge_r2)

# Lasso
lasso_best, lasso_rmse, lasso_r2 = evaluate_model("Lasso", lasso,
lasso_params)
results["Lasso"] = (lasso_rmse, lasso_r2)

# Decision Tree
dtr_best, dtr_rmse, dtr_r2 = evaluate_model("DecisionTree", dtr,
dtr_params)
results["DecisionTree"] = (dtr_rmse, dtr_r2)

# Random Forest
rfr_best, rfr_rmse, rfr_r2 = evaluate_model("RandomForest", rfr,
rfr_params)
results["RandomForest"] = (rfr_rmse, rfr_r2)

results
```

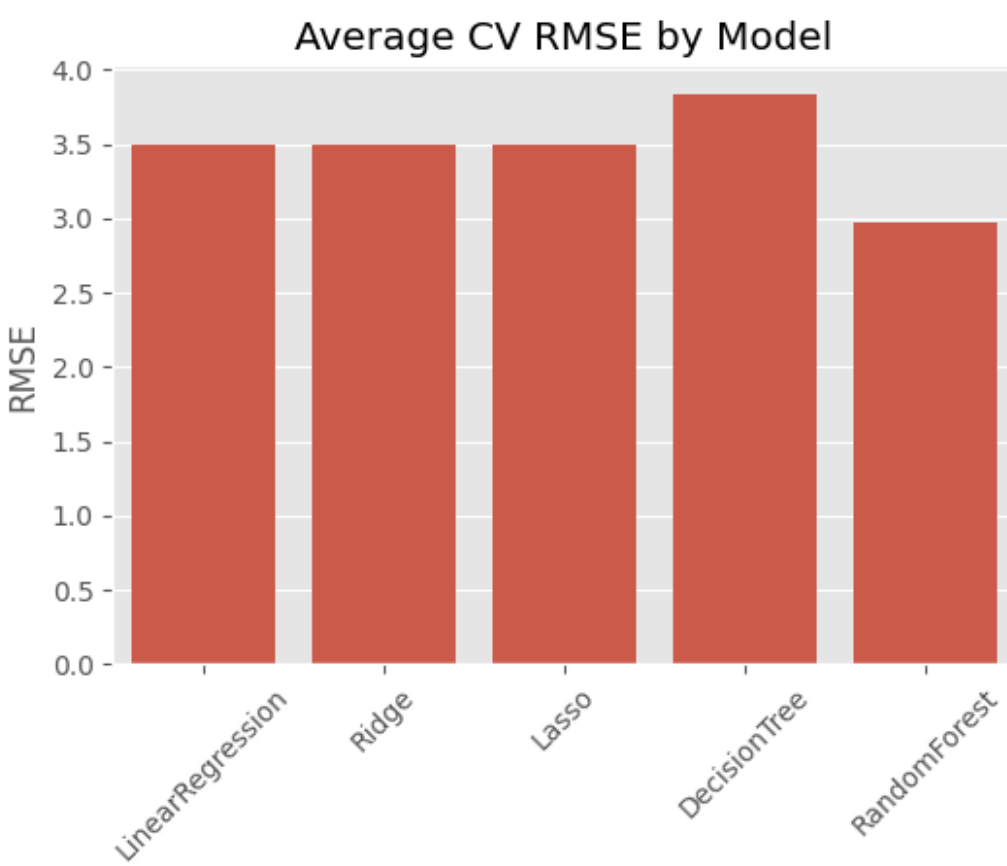
```
LinearRegression — Avg CV RMSE: 3.492, Avg CV R²: 0.795
Ridge — Avg CV RMSE: 3.489, Avg CV R²: 0.796
Lasso — Avg CV RMSE: 3.490, Avg CV R²: 0.796
DecisionTree — Avg CV RMSE: 3.836, Avg CV R²: 0.753
RandomForest — Avg CV RMSE: 2.968, Avg CV R²: 0.850
```

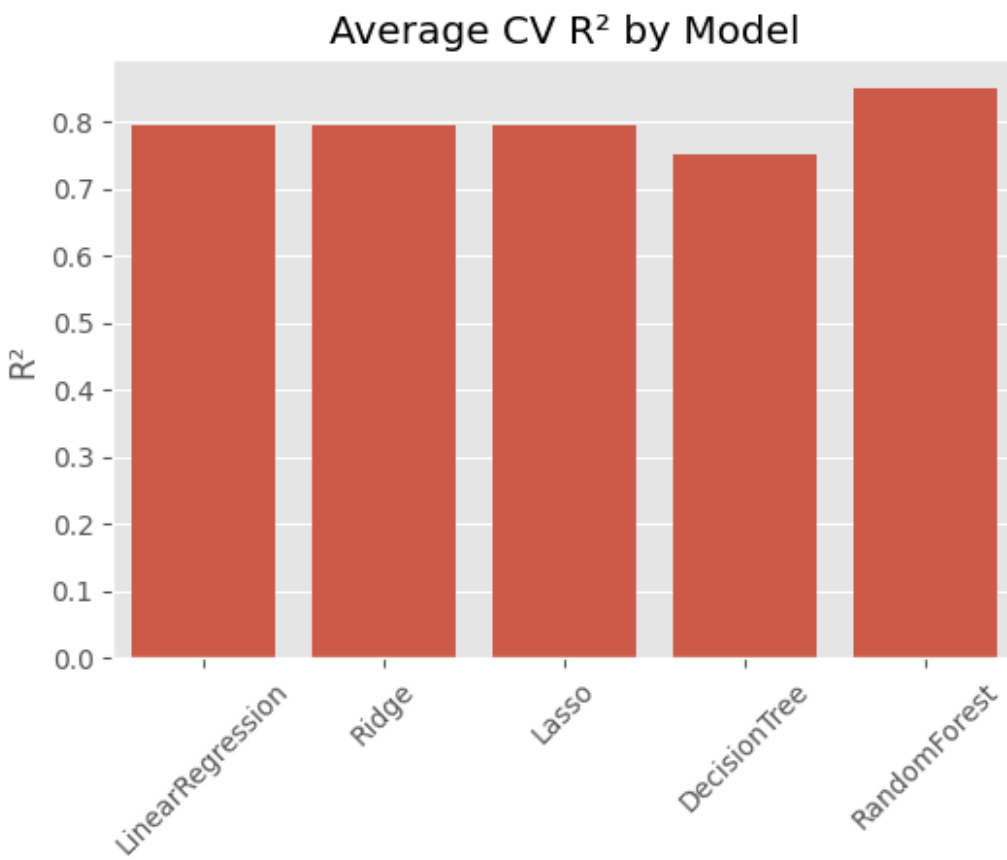
```
{'LinearRegression': (np.float64(3.492366176140289),
  np.float64(0.7954977225864415)),
 'Ridge': (np.float64(3.4889776888551167),
np.float64(0.7960328159993671)),
 'Lasso': (np.float64(3.4896206030644215),
np.float64(0.7961261335582112)),
 'DecisionTree': (np.float64(3.8358751965830087),
  np.float64(0.7525262881561728)),
```

```
 'RandomForest': (np.float64(2.9680912234543615),
  np.float64(0.8500791026473195))}
```

small comparison plot:

```
model_names = list(results.keys())
rmse_vals = [results[m][0] for m in model_names]
r2_vals = [results[m][1] for m in model_names]

plt.figure(figsize=(6,4))
sns.barplot(x=model_names, y=rmse_vals)
plt.title("Average CV RMSE by Model")
plt.ylabel("RMSE")
plt.xticks(rotation=45)
plt.show()

plt.figure(figsize=(6,4))
sns.barplot(x=model_names, y=r2_vals)
plt.title("Average CV R² by Model")
plt.ylabel("R²")
plt.xticks(rotation=45)
plt.show()
```





In this section I compared several regression algorithms under a consistent evaluation framework. I used 5-fold cross-validation to estimate performance for standard Linear Regression, Ridge and Lasso (which add regularisation), a Decision Tree Regressor, and a Random Forest Regressor, performing grid search for models with hyperparameters. Linear Regression, Ridge and Lasso all achieve similar performance, with average RMSE around 3.49 mpg and R^2 around 0.80, showing that regularisation does not dramatically improve over the baseline for this dataset. The Decision Tree performs slightly worse (higher RMSE and lower R^2), suggesting that a single tree is too simple and prone to over-fitting or instability. The Random Forest achieves the best results, with an average RMSE of about 2.97 mpg and R^2 of roughly 0.85, meaning it captures non-linearities and interactions more effectively. The bar charts of RMSE and R^2 make these differences easy to see and support choosing Random Forest as the main candidate for the final model.

1. Final Model: Random Forest Regressor

8.1 – Fit best Random Forest on full training set and evaluate on test set

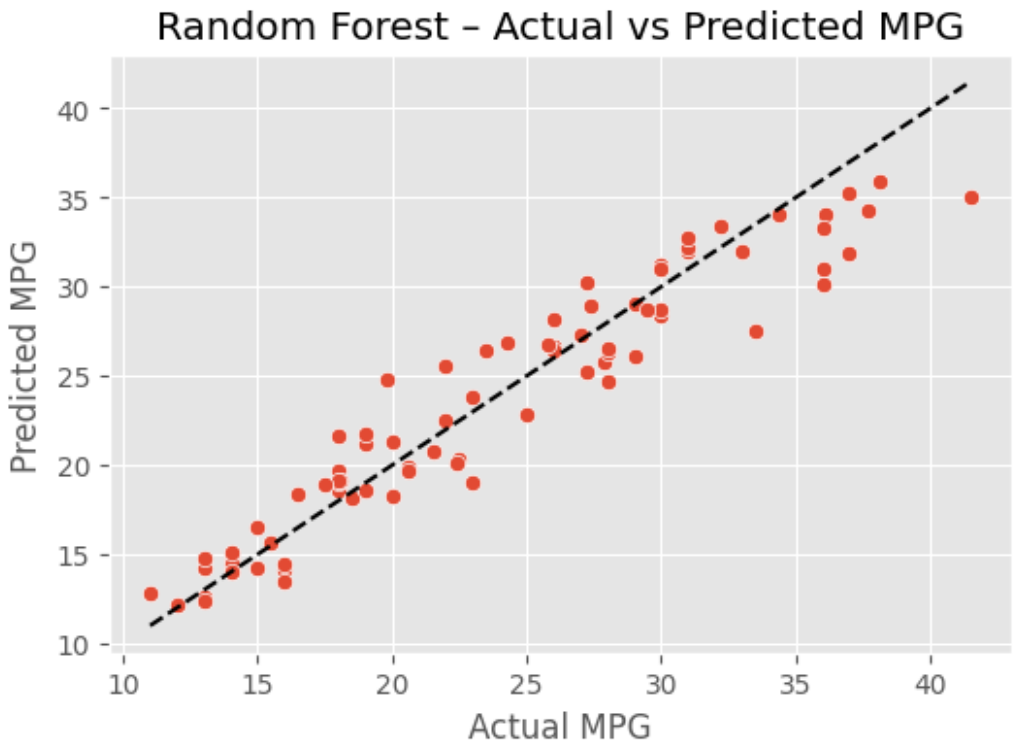
```
# ======================================================
# 8. FINAL RANDOM FOREST MODEL
# ======================================================

# rfr_best comes from Section 7 GridSearchCV
final_rf = rfr_best
```

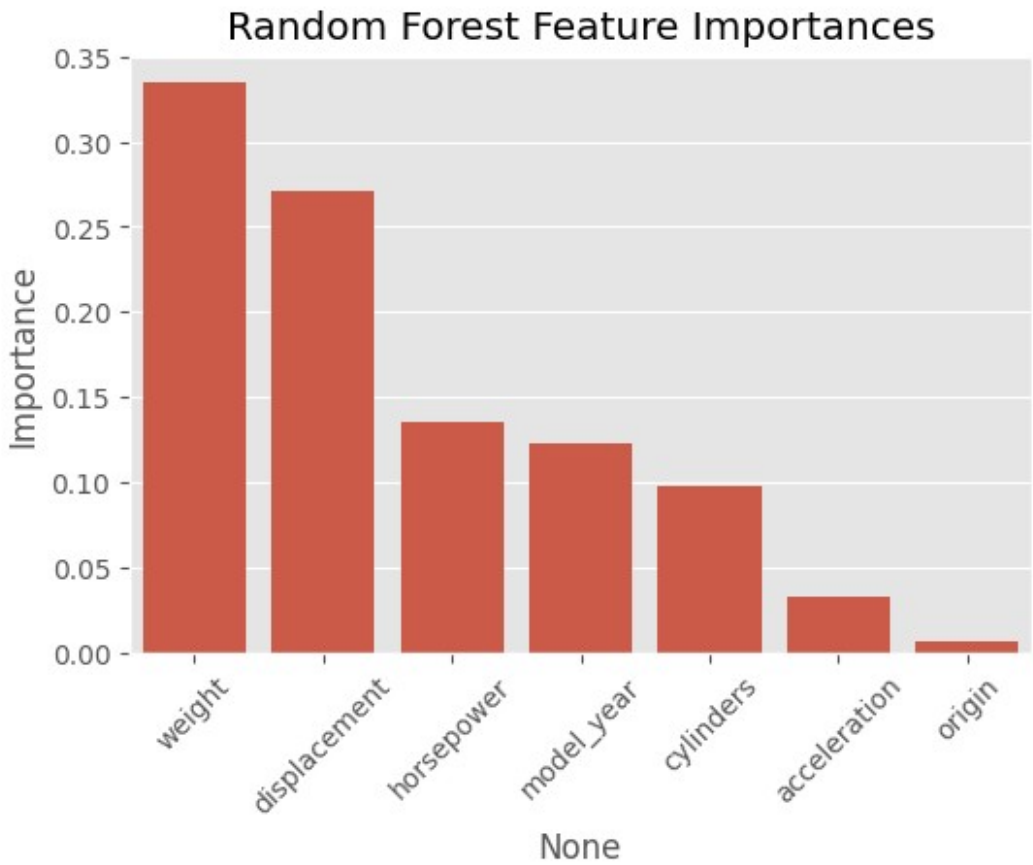
```
# Fit on full training data (if not already fitted)
final_rf.fit(X_train, y_train)

# Predict on test set
y_pred_rf = final_rf.predict(X_test)

# Metrics
rf_test_mse = mean_squared_error(y_test, y_pred_rf)
rf_test_rmse = np.sqrt(rf_test_mse)
rf_test_r2 = r2_score(y_test, y_pred_rf)

print("Random Forest — Test MSE:", rf_test_mse)
print("Random Forest — Test RMSE:", rf_test_rmse)
print("Random Forest — Test R²:", rf_test_r2)
```

```
Random Forest — Test MSE: 5.230879936708857
Random Forest — Test RMSE: 2.287111701843366
Random Forest — Test R²: 0.9117046934263627
```

8.2 – Predicted vs actual and feature importance

```
# Predicted vs actual plot
plt.figure(figsize=(6, 4))
sns.scatterplot(x=y_test, y=y_pred_rf)
plt.plot([y_test.min(), y_test.max()],
         [y_test.min(), y_test.max()],
         color="black", linestyle="--")
plt.xlabel("Actual MPG")
plt.ylabel("Predicted MPG")
plt.title("Random Forest — Actual vs Predicted MPG")
plt.show()

# Feature importance
importances = pd.Series(final_rf.feature_importances_,
index=features_full)
importances = importances.sort_values(ascending=False)

plt.figure(figsize=(6, 4))
sns.barplot(x=importances.index, y=importances.values)
plt.title("Random Forest Feature Importances")
plt.ylabel("Importance")
plt.xticks(rotation=45)
plt.show()

importances
```


Random Forest – Actual vs Predicted MPG
Predicted MPG
Actual MPG
10
15
20
25
30
35
40


Random Forest Feature Importances
Importance
0.00
0.05
0.10
0.15
0.20
0.25
0.30
0.35
weight
displacement
horsepower
model_year
cylinders
acceleration
origin
None

```
weight          0.334460
displacement    0.270989
horsepower      0.135145
model_year      0.122980
cylinders       0.097350
acceleration    0.032262
origin          0.006813
dtype: float64
```

After identifying Random Forest as the best-performing algorithm, I fitted the best-found Random Forest model on the full training set and evaluated it on the test set. The initial tuned Random Forest reaches a test RMSE of about 2.29 mpg and an R^2 of approximately 0.91, which is a substantial improvement over the baseline linear model. The actual vs predicted scatter plot shows points tightly clustered around the diagonal line, indicating that the model's predictions closely match the true mpg values across the full range. The feature importance plot reveals that weight and displacement are the most influential predictors, followed by horsepower and model_year, while origin and acceleration contribute relatively little. This aligns with domain intuition: heavier cars with larger engines tend to use more fuel, and newer models are more efficient. Overall, these results suggest that the Random Forest model is well-suited for predicting fuel efficiency from automotive attributes.

8.3 Additional Hyperparameter Tuning

```
rf = RandomForestRegressor(random_state=21)

rf_params_refined = {
    "n_estimators": [300, 500, 800],
    "max_depth": [8, 10, 12],
    "min_samples_leaf": [1, 2],
    "max_features": ["auto", "sqrt", 0.5]
}

rf_grid = GridSearchCV(
    rf,
    param_grid=rf_params_refined,
    cv=5,
    scoring="neg_root_mean_squared_error",
    n_jobs=-1
)
rf_grid.fit(X_train, y_train)

best_rf = rf_grid.best_estimator_
best_rf_rmse = -rf_grid.best_score_
best_rf_r2 = cross_val_score(
    best_rf, X_train, y_train,
    cv=5, scoring="r2"
).mean()
```

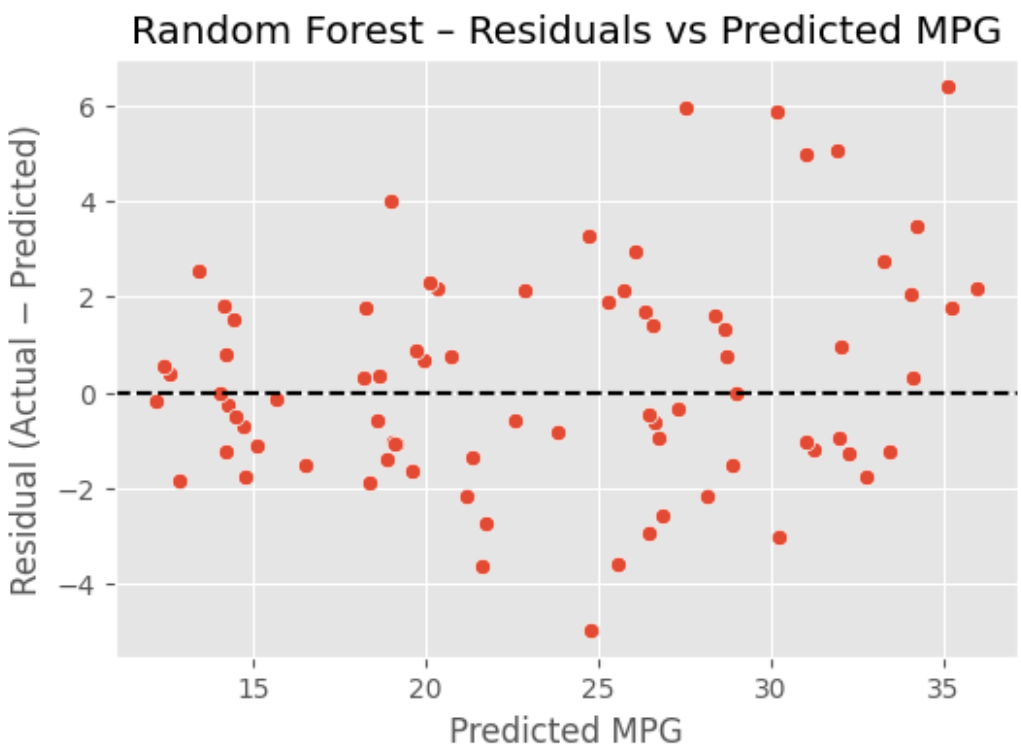
```python
print("Best Random Forest parameters:", rf_grid.best_params_)
print("Best CV RMSE:", best_rf_rmse)
print("Best CV R²:", best_rf_r2)
```

```
Best Random Forest parameters: {'max_depth': 10, 'max_features': 0.5,
'min_samples_leaf': 1, 'n_estimators': 300}
Best CV RMSE: 2.954823282735063
Best CV R²: 0.8517231692327962
```

```python
best_rf.fit(X_train, y_train)
y_pred_best = best_rf.predict(X_test)

best_test_mse = mean_squared_error(y_test, y_pred_best)
best_test_rmse = np.sqrt(best_test_mse)
best_test_r2 = r2_score(y_test, y_pred_best)

print("Refined RF — Test RMSE:", best_test_rmse)
print("Refined RF — Test R²:", best_test_r2)
```

```
Refined RF — Test RMSE: 2.276529629274703
Refined RF — Test R²: 0.9125198576429948
```

```python
residuals = y_test - y_pred_rf

plt.figure(figsize=(6, 4))
sns.scatterplot(x=y_pred_rf, y=residuals)
plt.axhline(0, color="black", linestyle="--")
plt.xlabel("Predicted MPG")
plt.ylabel("Residual (Actual - Predicted)")
plt.title("Random Forest — Residuals vs Predicted MPG")
plt.show()
```



In the final step I carried out a more detailed hyperparameter search for the Random Forest, exploring combinations of the number of trees, maximum depth, minimum samples per leaf and the number of features considered at each split. The refined model slightly improves cross-validated RMSE to about 2.95 mpg and R^2 to around 0.85 on the training folds, and achieves a test RMSE of roughly 2.28 mpg with an R^2 of about 0.91. This confirms that the Random Forest's performance is stable and that further tuning yields only marginal gains, suggesting the model is close to its optimal complexity for this dataset. The residuals vs predicted plot shows residuals mostly between −4 and +4 mpg with no strong systematic trend, which indicates that the model errors are approximately random and that there is no obvious remaining structure that the model fails to capture. This supports using the refined Random Forest as the final predictive model for fuel efficiency.

Conclusion

This project set out to answer the research question: **How accurately can car fuel efficiency (mpg) be predicted from technical attributes such as cylinders, displacement, horsepower, weight, acceleration, model year and origin, and which machine learning model provides the best predictive performance?** Using the cleaned UCI Auto MPG dataset, several regression models were trained and compared, including Linear Regression, Ridge, Lasso, Decision Tree and Random Forest. The baseline linear model already explained around 80% of the variance in mpg, but the Random Forest Regressor consistently achieved the best performance, with a cross-validated RMSE of about 3.0 mpg and a test RMSE of roughly 2.3 mpg, corresponding to an (R^2) close to 0.91. These results show that fuel efficiency can be predicted with fairly high accuracy from the available attributes, especially when non-linear relationships and interactions are captured by ensemble tree methods. Feature importance analysis confirms that **weight**, **displacement** and **horsepower** are the dominant drivers of fuel consumption, followed by

model_year, which reflects technological improvements over time, while origin and acceleration add only minor incremental value. Overall, the findings indicate that manufacturers can use models of this type to estimate fuel efficiency for new vehicle designs early in the development process, quantitatively assess the impact of design choices, and support more informed decisions aimed at improving fuel economy.